

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева»
(Самарский университет)

Институт _____ информатики и кибернетики
Кафедра _____ программных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**«РАЗРАБОТКА СИСТЕМЫ СБОРА И АНАЛИЗА ИНФОРМАЦИИ О
НОВОЙ КОРОНАВИРУСНОЙ ИНФЕКЦИИ»**

по направлению подготовки 02.03.02

Фундаментальная информатика и информационные технологии
(уровень бакалавриата)

направленность (профиль) «Информационные технологии»

Обучающийся _____ И.И. Иванов
(подпись, дата)

Руководитель ВКР

доцент каф. ПС _____ Д.А. Попова-Коварцева
(подпись, дата)

Нормоконтролер _____ Е.В. Сопченко
(подпись, дата)

Самара 2022

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева»
(Самарский университет)

Институт _____ информатики и кибернетики
Кафедра _____ программных систем

УТВЕРЖДАЮ
Заведующий кафедрой
_____ С.В. Востокин
« ____ » _____ 2021 г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
(бакалавр)

обучающемуся _____ Иванову Ивану Ивановичу
группа _____ 6414-020302D

Тема работы: _____ Разработка системы сбора и анализа информации
о новой коронавирусной инфекции
Исходные данные _____ см. в приложении к заданию

Структурные части работы (перечень вопросов, подлежащих разработке):

- 1) Провести анализ предметной области: COVID-19, иммунитет от COVID-19, коллективный иммунитет, вакцинация от коронавируса, меры по борьбе с коронавирусной инфекцией, эффективность принимаемых мер
- 2) Сделать обзор систем-аналогов в области анализа медицинских данных
- 3) Разработать структурную схему системы
- 4) Разработать информационно-логический проект системы по методологии UML
- 5) Разработать интерфейс пользователя
- 6) Разработать и реализовать программное и информационное обеспечение, провести тестирование и отладку
- 7) Провести ресурсные расчеты

Научный руководитель
доцент, кафедра программных систем

Задание принял к исполнению

_____ (Д.А. Попова-Коварцева)
« 25 » октября 2021 г.

_____ (И.И. Иванов)
« 25 » октября 2021 г.

ПРИЛОЖЕНИЕ

к заданию на выпускную квалификационную работу бакалавра
обучающемуся И.И. Иванову группы № 6414-020302D

Тема: «Разработка системы сбора и анализа информации о новой
коронавирусной инфекции»

Исходные данные к работе:

1 Характеристики объекта автоматизации:

1) объект автоматизации – процесс сбора и анализа информации
о новой коронавирусной инфекции;

2) виды автоматизируемой деятельности:

– процесс получения актуальной информации о
распространении коронавирусной инфекции;

– процесс получения актуальной информации об
ограничениях, действующих в регионах России;

– процесс визуализации данных о распространении
коронавирусной инфекции в виде графиков;

– процесс формирования текстовых отчетов по
коронавирусной инфекции в регионах России;

3) количество ролей – 2;

4) минимальная длина отчетного периода – 1 день;

5) максимальная длина названия региона – 100 символов;

6) язык интерфейса – русский.

2 Требования к информационному обеспечению:

1) Информационное обеспечение разрабатывается на основании
следующих литературных источников:

– Программная инженерия [Электронный ресурс]. URL:
[http://repo.ssau.ru/bitstream/Uchebnye-izdaniya/Programmnaya-inzheneriya-Elektronnyi-resurs-elektron-uchebmetod-kompleks-po-discipline-v-LMS-Moodle-](http://repo.ssau.ru/bitstream/Uchebnye-izdaniya/Programmnaya-inzheneriya-Elektronnyi-resurs-elektron-uchebmetod-kompleks-po-discipline-v-LMS-Moodle-71103/1/Зеленко%20Л.%20С.%20Программная.pdf)

71103/1/Зеленко%20Л.%20С.%20Программная.pdf

(дата

обращения 28.03.2022)

– PostgreSQL. Основы языка SQL.: учеб. пособие / Е.П. Моргунов; под. ред. Е.В. Рогова, П.В. Лузанова. – СПб.: БХВ-Петербург, 2018. – 336 с.: ил.;

2) Структура базы данных разрабатывается на основании следующих сведений:

- о регионах России (название);
- об ограничениях (описание);
- о региональной статистике распространения коронавируса (регион, дата, количество новых случаев, количество госпитализированных, количество выздоровевших, количество умерших, количество вакцинированных первым компонентом, количество полностью вакцинированных, уровень коллективного иммунитета);
- о региональных ограничениях (регион, дата, действующее ограничение).

3) Обеспечить контроль целостности базы данных.

3 Требования к техническому обеспечению:

3.1 Требования к техническому обеспечению серверной части:

- 1) тип ЭВМ – IBM PC совместимый;
- 2) объем ОЗУ – не менее 2 Гб;
- 3) объем свободного пространства на внешнем диске – не менее 50 Гб;
- 4) наличие подключения к сети Интернет;
- 5) манипулятор – мышь;
- 6) технические характеристики определяются в процессе выполнения работы.

3.2 Требования к техническому обеспечению клиентской части:

- 1) тип ЭВМ – IBM PC совместимый;
- 2) монитор с разрешающей способностью не ниже

800x600;

3) манипулятор – мышь;

4) технические характеристики определяются в процессе выполнения работы.

4 Требования к программному обеспечению:

4.1 Требования к программному обеспечению серверной части:

1) тип операционной системы – Windows 7 и выше;

2) СУБД – PostgreSQL 14;

4.2 Требования к программному обеспечению клиентской части:

1) тип операционной системы – Windows 7 и выше;

2) браузер – Google Chrome 86.0.4240.183 (64-битный) и выше, Firefox 83.0 (64-битный) и выше.

4.3 Требования к программному обеспечению рабочего места разработчика:

1) тип операционной системы – Windows 7 и выше;

2) язык программирования – Python;

3) среда программирования – PyCharm 2020.1.5;

4) СУБД – PostgreSQL 14;

5) среда проектирования – StarUML 4.0.0.

5 Общие требования к проектируемой системе.

5.1 Функции, реализуемые системой:

1) Общесистемные функции:

- ведение базы данных;
- выдача актуальной информации о распространении коронавирусной инфекции;
- выдача исторической информации о коронавирусной инфекции в России;
- формирование аналитических отчетов в формате документа Word;

- выдача информации о распространении коронавирусной инфекции в формате JSON.

2) Функции пользователя:

- просмотр актуальной информации о коронавирусе в России;
- просмотр исторических данных о коронавирусе;
- скачивание актуальной информации в формате JSON;
- установка параметров (регион, дата) для просмотра исторической информации;
- установка параметров (регион, даты, включение анализа мер) для формирования аналитического отчета.

3) Функции администратора:

- авторизация администратора:
 - а) ввод логина;
 - б) ввод пароля;
- запуск процедуры сбора актуальной информации;
- редактирование списка регионов;
- редактирование списка ограничений.

5.2 Технические требования к системе:

- 1) режим работы – диалоговый;
- 2) температура окружающего воздуха – 15-25°C;
- 3) влажность окружающего воздуха – 45-75%;
- 4) система должна удовлетворять санитарным правилам и нормам СанПин 2.2.2/2.4.2198-07;
- 5) условия работы средств вычислительной техники должны соответствовать ГОСТ 12.1.005, 12.1.007.

Научный руководитель,

к.т.н., доцент

Д.А. Попова-Коварцева

подпись, дата

РЕФЕРАТ

Пояснительная записка: 73 с, 29 рисунков, 12 таблиц, 27 источников, 2 приложения.

Графическая часть: 19 слайдов презентации PowerPoint.

КОРОНАВИРУСНАЯ ИНФЕКЦИЯ, СБОР ДАННЫХ, АНАЛИЗ ДАННЫХ, ВИЗУАЛИЗАЦИЯ ДАННЫХ, ОТЧЕТНАЯ СИСТЕМА, БАЗА ДАННЫХ

Цель работы – разработка автоматизированной системы сбора и анализа информации о новой коронавирусной инфекции.

В процессе работы были разработаны алгоритмы и соответствующая программа, позволяющая пользователю просматривать и скачивать актуальную и историческую информацию о распространении коронавирусной инфекции в регионах России, а также формировать аналитические отчеты. Система позволяет в автоматическом режиме собирать официальные данные по коронавирусу в России.

Система разработана на языке Python с использованием фреймворка Django, библиотек Beautiful Soup, python-docx, Matplotlib, Pillow, Psycopg 2 и функционирует под управлением операционных систем Windows 7/8/10. Доступ к данным осуществляется с помощью СУБД PostgreSQL 14.

СОДЕРЖАНИЕ

Введение	10
1 Описание и анализ предметной области	11
1.1 Основные понятия и определения	11
1.1.1 COVID-19	11
1.1.2 Иммуитет от COVID-19	11
1.1.3 Коллективный иммунитет	11
1.1.4 Вакцинация от коронавируса	12
1.1.5 Меры по борьбе с коронавирусной инфекцией	13
1.1.6 Эффективность принимаемых мер	14
1.2 Обзор систем-аналогов	14
1.2.1 Стопкоронавирус.рф	14
1.2.2 Statistica	15
1.2.3 MedCalc	16
1.3 Постановка задачи	17
2 Проектирование системы	19
2.1 Выбор и обоснование архитектуры системы	19
2.2 Структурная схема системы	19
2.3 Разработка прототипа интерфейса системы	21
2.4 Разработка информационно-логического проекта системы	24
2.4.1 Язык UML	24
2.4.2 Диаграмма вариантов использования	24
2.4.3 Диаграмма классов	26
2.4.4 Сценарий	27
2.4.5 Диаграмма состояний	30
2.4.6 Диаграмма деятельности	32
2.4.7 Диаграмма последовательности	34
2.4.8 Логическая модель данных системы	36
2.5 Выбор и обоснование комплекса программных средств	39
2.5.1 Выбор языка программирования	39

2.5.2 Выбор среды программирования	39
2.5.3 Выбор СУБД.....	39
3 Реализация системы	41
3.1 Разработка и описание интерфейса пользователя	41
3.2 Диаграммы реализации.....	44
3.2.1 Диаграмма развертывания	44
3.2.2 Диаграмма компонентов	45
3.2.3 Диаграмма классов системы.....	46
3.2.4 Физическая схема данных	47
3.3 Выбор и обоснование комплекса технических средств	49
3.3.1 Расчет объема занимаемой памяти	49
3.3.2 Минимальные требования, предъявляемые к серверу	51
3.3.3 Минимальные требования, предъявляемые к клиенту	51
Заключение	52
Список использованных источников	53
Приложение А. Руководство пользователя	57
А.1 Назначение системы.....	57
А.2 Условия работы системы	57
А.3 Работа с системой.....	58
А.3.1 Аналитические отчеты.....	58
А.3.2 Региональная статистика	59
А.3.3 Информация о сайте.....	60
Приложение Б. Код программы.....	61

ВВЕДЕНИЕ

В январе 2020 года Всемирная организация здравоохранения объявила вспышку эпидемии заболевания COVID-19 чрезвычайной ситуацией международного значения, а 11 марта того же года назвала происходящие события пандемией [1,2]. К тому моменту вспышка заболеваемости уже происходила в Европе и привела почти к 2000 смертей.

Чтобы предотвратить новые потенциальные жертвы, правительства начали вводить различные меры, направленные на снижение новых случаев заражения. Во многих странах был введен режим чрезвычайной ситуации, вводились ограничения различной степени строгости. Закрытие границ, ограничение работы общественных пространств, введение режима самоизоляции – существует большое количество различных методов, которые использовались для ограничения пандемии.

На данный момент распространение COVID-19 все еще является серьезным испытанием для человечества. Несмотря на появление вакцин и активную работу над поиском новых препаратов, каждый день в мире регистрируются сотни тысяч новых случаев заражения и тысячи смертей. К тому же, вирус мутирует, появляются новые штаммы, имеющие большую заразность и устойчивость к вакцинам.

Актуальность выбранной темы состоит в том, что для эффективного создания перечня мер противодействия COVID-19 стоит опереться на уже существующие данные о распространении и мерах борьбы с пандемией.

Во время выполнения выпускной квалификационной работы необходимо разработать автоматизированную систему, предназначенную для сбора и анализа информации о новой коронавирусной инфекции.

1 Описание и анализ предметной области

Предметная область – это часть реального мира, которая подлежит изучению с целью автоматизации организации управления. Предметной областью информационной системы является совокупность объектов, свойства которых и отношения, между которыми представляют интерес для пользователей информационных систем (ИС) [3].

1.1 Основные понятия и определения

1.1.1 COVID-19

COVID-19 – это потенциально тяжелое острое инфекционное респираторное заболевание, вызываемое коронавирусом SARS-CoV-2 [4].

SARS-CoV-2 (Severe acute respiratory syndrome-related coronavirus 2) – оболочечный одноцепочный (+)РНК-вирус, вызывающий COVID-19 [5].

1.1.2 Иммунитет от COVID-19

Иммунитет (лат. *immunitas* – освобождение, избавление от чего-либо) – невосприимчивость организма к инфекционным и неинфекционным агентам и веществам; способность организма специфически реагировать на введение генетически чужеродных веществ (антигенов) [6].

Иммунитет может появиться у человека после перенесенной болезни либо вакцинации.

24 мая 2021 года в журнале *Nature* было опубликовано исследование, проведенное иммунологами Университета Вашингтона в Сент-Луисе (штат Миссури). Исследователи пришли к выводу, что перенесение болезни индуцирует устойчивую и продолжительную иммунную память в организме человека [7].

1.1.3 Коллективный иммунитет

«Коллективный иммунитет», известный также как «популяционный иммунитет», является косвенной защитой от инфекционного заболевания,

которая возникает благодаря развитию иммунитета у населения либо в результате вакцинации, либо в результате перенесенной ранее инфекции [8].

Для каждого отдельного заболевания существует свой порог коллективного иммунитета, позволяющий защитить общество от его распространения. По оценке российского иммунолога Николая Крючкова, для победы над COVID-19 необходим уровень коллективного иммунитета 85% [9].

1.1.4 Вакцинация от коронавируса

Вакцина – (лат. *vaccinus* – коровий) медицинский препарат, получаемый из убитых или живых, но ослабленных микробов-возбудителей инфекционных болезней или продуктов их жизнедеятельности. Применяется для иммунизации человека и животных с профилактической (вакцинация) или лечебной (вакциноterapia) целями [10];

По состоянию на 23 марта 2021 года существует 15 различных вакцин, зарегистрированных или одобренных как минимум одним национальным регулятором, а также несколько десятков вакцин, находящихся в стадии разработки.

Эффективность вакцины – относительное снижение риска заболевания у привитых по сравнению с непривитыми [11]. Эффективность основных вакцин, используемых в мире на данный момент, представлена в таблице 1.

Таблица 1 – Показатели эффективности основных используемых вакцин [11].

Вакцина	Эффективность
Pfizer–BioNTech BNT162b2 mRNA	95%
Moderna–US National Institutes of Health mRNA-1273	94%
Gamaleya GamCovidVac (Спутник V)	91%
Johnson & Johnson Ad26.COV2.S	67%
AstraZeneca-Oxford ChAdOx1 nCov-19	67 %

1.1.5 Меры по борьбе с коронавирусной инфекцией

Меры по борьбе с коронавирусной инфекцией – это ряд действий и ограничений, направленных на снижение распространения заболевания, смертности, борьбу с экономическими и социальными кризисами сопутствующими пандемии. Существуют на различных уровнях – федеральные, региональные, городские, персональные. В данной работе будут рассмотрены федеральные и региональные меры. Рассмотрим некоторые примеры методов противодействия пандемии, принятые в Российской Федерации:

– высшим должностным лицам (руководителям высших исполнительных органов государственной власти) субъектов Российской Федерации с учетом положений настоящего Указа, исходя из санитарно-эпидемиологической обстановки и особенностей распространения новой коронавирусной инфекции (COVID-19) в субъекте Российской Федерации, обеспечить ... приостановление (ограничение, в том числе путем определения особенностей режима работы, численности работников) деятельности находящихся на соответствующей территории отдельных организаций независимо от организационно-правовой формы и формы собственности, а также индивидуальных предпринимателей с учетом методических рекомендаций Федеральной службы по надзору в сфере защиты прав потребителей и благополучия человека, рекомендаций главных государственных санитарных врачей субъектов Российской Федерации [12];

– Минпромторг России заявил, что в отдельных субъектах РФ вводятся ограничения работы фуд-корт в торговых центрах. Альтернативой может быть четко установленный регламент их работы на время ухудшения ситуации с распространением новой коронавирусной инфекции [13];

– в случае принятия решения о приостановлении (ограничении) деятельности отдельных организаций и ИП, за работниками таких организаций и ИП сохраняется заработная плата;

– Роспотребнадзор отметил, что справка об отсутствии коронавирусной инфекции представляет собой специальный медицинский документ, защищенный от подделок специальным QR-кодом. Этот код уникален и присваивается только одной справке. При подозрении подлинности представляемой справки с результатами теста на COVID-19 необходимо обратиться в правоохранительные органы [13].

1.1.6 Эффективность принимаемых мер

В зависимости от того, на что нацелена конкретная мера противодействия, необходимо оценивать ее по различным метрикам. Так, например, эффективность ношения масок следует оценивать по динамике новых случаев заражения, в то время как на процент смертельных случаев среди заболевших эта мера влиять не будет. Эффективными будут считаться те меры или комплексы мер, которые оказали статистически значимое влияние на течение пандемии на территории принятия.

1.2 Обзор систем-аналогов

В настоящее время существует ряд программ, позволяющих собирать и анализировать данные медицинской статистики. Рассмотрим основные характеристики систем-аналогов.

1.2.1 Стопкоронавирус.рф

Стопкоронавирус.рф – это веб-сайт, созданный при поддержке Правительства Российской Федерации, и содержащий информацию о коронавирусной инфекции в России и мире.

К достоинствам данной системы относятся:

- регулярно обновляемая официальная информация;
- большое количество данных;
- простота получения информации для рядового пользователя.

К недостаткам данной системы относятся:

- отсутствие исторических данных;

— невозможность скачать данные.

На рисунке 1 представлена страница сайта с оперативными региональными данными по состоянию на 19 мая 2022 года.

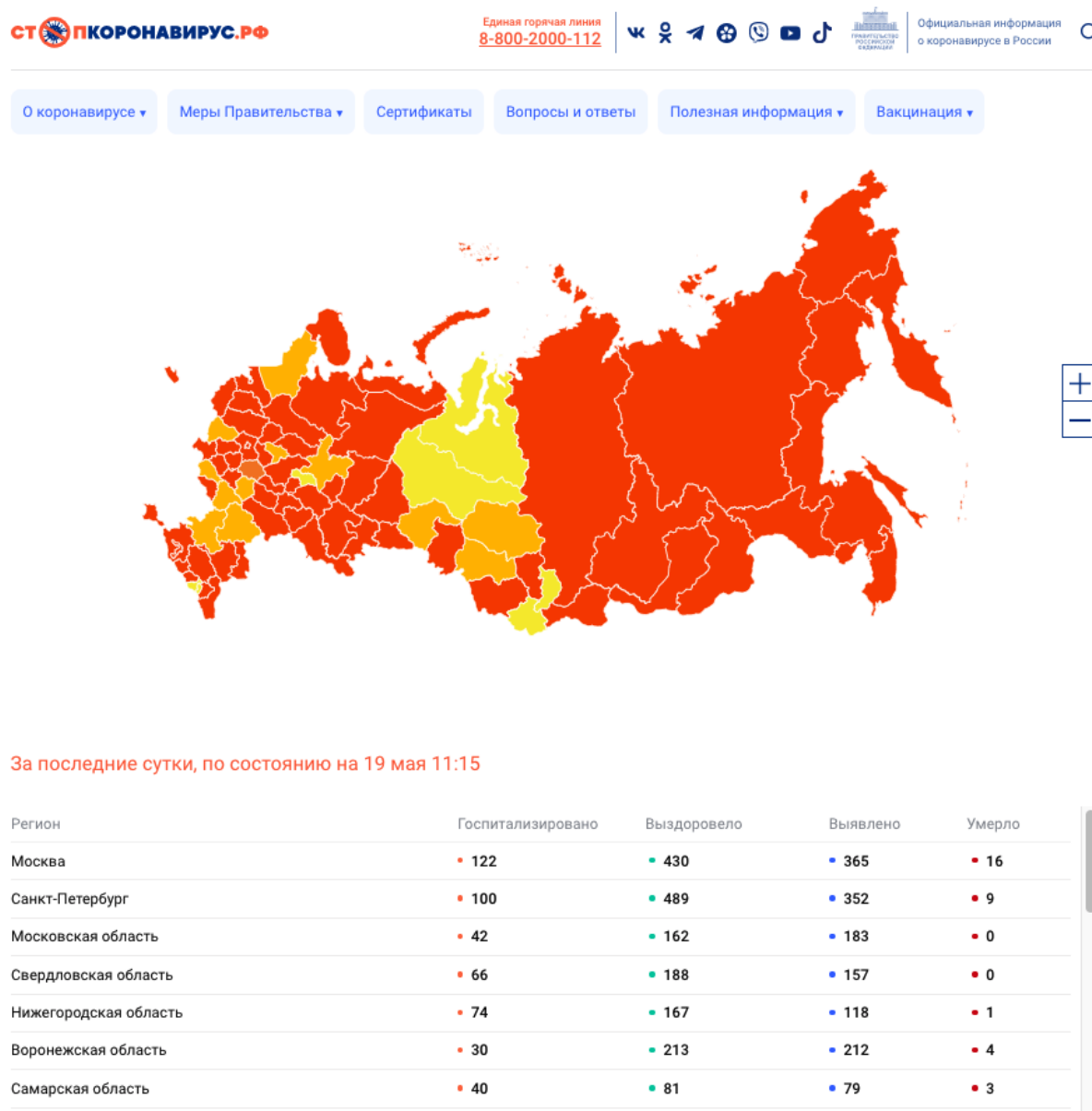


Рисунок 1 – Стопкоронавирус.рф, страница оперативной информации

1.2.2 Statistica

Statistica — это набор аналитических программных продуктов и решений, разработанный компанией StatSoft. Программное обеспечение включает в себя набор процедур анализа данных, управления данными, визуализации данных и интеллектуального анализа данных; а также

различные методы прогнозного моделирования, кластеризации, классификации и исследования [14].

На рисунке 2 представлен интерфейс программы Statistica.

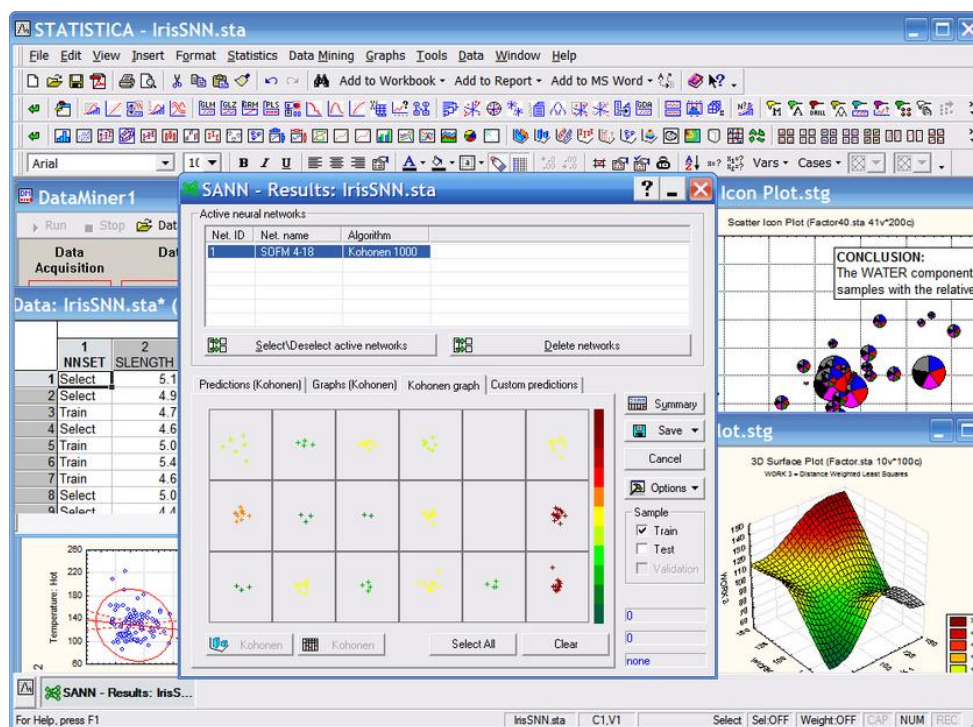


Рисунок 2 – Интерфейс программы Statistica

Данная система обладает следующими преимуществами:

- большой функционал, позволяющий найти необходимый аналитический инструмент для решения практически любой задачи;
- возможность интеграции со средой программирования языка R, широко используемого в статистической обработке данных, что позволяет реализовать дополнительные методы.

К недостаткам можно отнести тот факт, что Statistica предназначена для опытных пользователей, и начинающие аналитики могут найти количество функций программы чрезмерным.

1.2.3 MedCalc

MedCalc – это пакет аналитического программного обеспечения, ориентированный на ученых, специализирующихся в биомедицинской отрасли. Он включает в себя более 220 статистических тестов, процедур и

графиков, возможность анализа ROC-кривых, средства сравнения методов и контроля качества [15]. Интерфейс программы представлен на рисунке 3.

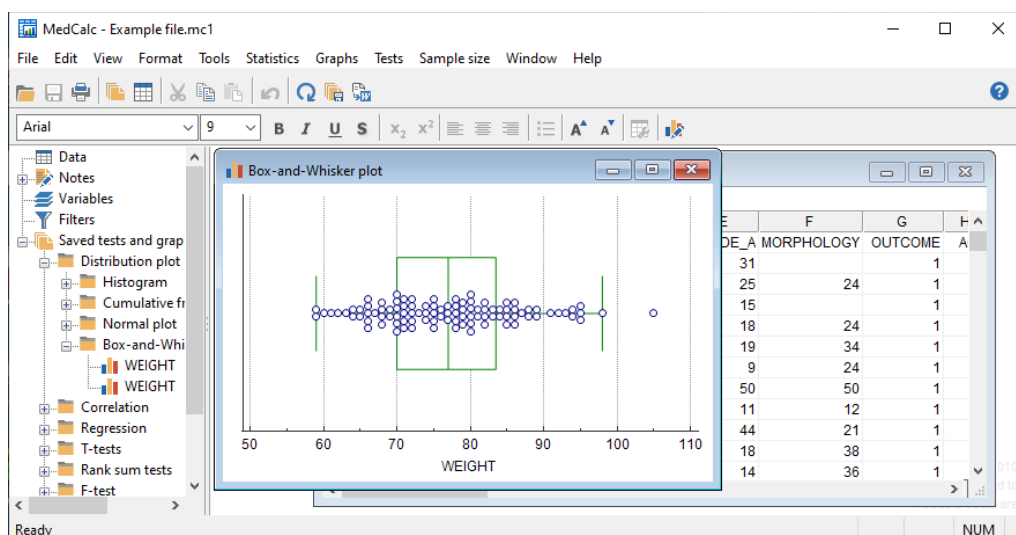


Рисунок 3 – Интерфейс программы MedCalc

Функциональной особенностью данной системы является возможность работы только в графическом интерфейсе, без использования командной строки. Это уменьшает возможности системы, но делает ее проще для пользователя.

1.3 Постановка задачи

Во время выпускной квалификационной работы необходимо разработать автоматизированную систему сбора и анализа информации о новой коронавирусной инфекции. Данная система будет осуществлять заполнение и ведение базы данных с информацией о коронавирусе в России, будет обладать методами выдачи актуальных и исторических данных о распространении COVID-19 и принимаемых мерах противодействия. С помощью данной системы пользователь сможет просматривать статистику распространения коронавируса, скачивать актуальные данные в формате JSON, получать исторические данные по выбранным регионам и датам, а также формировать текстовые отчеты в формате документов Word.

Таким образом, система должна решать следующие задачи:

- 1) Общесистемные функции:
 - ведение базы данных;

- выдача актуальной информации о распространении коронавирусной инфекции;
- выдача исторической информации о коронавирусной инфекции в России;
- формирование аналитических отчетов в формате документа Word;
- выдача информации о распространении коронавирусной инфекции в формате JSON.

2) Функции пользователя:

- просмотр актуальной информации о коронавирусе в России;
- просмотр исторических данных о коронавирусе;
- скачивание актуальной информации в формате JSON;
- установка параметров (регион, дата) для просмотра исторической информации;
- установка параметров (регион, даты, включение анализа мер) для формирования аналитического отчета.

3) Функции администратора:

- авторизация администратора:
 - 1) ввод логина;
 - 2) ввод пароля;
- запуск процедуры сбора актуальной информации;
- редактирование списка регионов;
- редактирование списка ограничений.

2 Проектирование системы

2.1 Выбор и обоснование архитектуры системы

Архитектура программного обеспечения определяется рекомендуемой практикой как фундаментальная организация системы, воплощенная в его компоненты, их отношения друг к другу и среде, а также принципы ее проектирования и эволюция [16].

Существуют различные виды архитектур, далее рассмотрены несколько основных.

1) локальная – при такой архитектуре не требуется обмен данными, все вычисления и необходимые данные находятся на устройстве пользователя;

2) файл-серверная – при такой архитектуре устройство пользователя соединено с файл-сервером по сети. Файл-сервер – компьютер, хранящий данные в файлах;

3) клиент-серверная – при такой архитектуре сервер не только хранит файлы, но и реализует часть функционала программной системы, например, выполняет процедуры СУБД. Такие системы сложны в разработке и поддержании работоспособности, однако позволяют разгрузить сеть и поддерживать непротиворечивость данных за счет централизованной обработки их сервером.

Так как разрабатываемая система должна регулярно пополняться актуальной информацией из Интернета, а также быть доступной широкому кругу пользователей, для разработки была выбрана клиент-серверная архитектура. Это позволит на стороне сервера обновлять и поддерживать правильность информации и не потребует установки локальной программы и дополнительных компонентов. Система будет реализована в виде веб-сайта.

2.2 Структурная схема системы

Система – множество элементов, находящихся в отношениях и связях друг с другом, которое образует определённую целостность, единство [17].

Если перенести это общее определение на информационные системы (ИС), то элементами в ней будут различные функциональные элементы – подсистемы, классы, функции и т.п., связанная работа которых будет решать поставленную перед ИС задачу. Сущность структурного подхода к разработке ИС заключается в ее декомпозиции (разбиении) на автоматизируемые функции: система разбивается на функциональные подсистемы, которые в свою очередь делятся на подфункции, подразделяемые на задачи и так далее.

На рисунке 4 представлена структурная схема системы.



Рисунок 4 – Структурная схема системы

В состав данной системы входят следующие подсистемы:

- 1) подсистема работы с базой данных, которая отвечает за взаимодействие с базой данных системы, получение необходимой информации и поддержание целостности информации. В состав данной подсистемы входит подсистема обновления базы данных, которая отвечает за регулярное получение актуальной информации;
- 2) подсистема формирования отчетов, которая отвечает за создание текстовых документов отчетов по заданной пользователем информации;
- 3) справочная подсистема, отвечающая за выдачу пользователю справочной информации;

4) подсистема формирования страниц, которая отвечает за формирование HTML-страниц сайта и наполнение их информацией;

5) подсистема администрирования, которая отвечает за управление сайтом администратором;

6) подсистема управления, которая отвечает за взаимодействие остальных подсистем и взаимодействие пользователя с сайтом.

2.3 Разработка прототипа интерфейса системы

Интерфейс – компоненты интерактивной системы (программное или аппаратное обеспечение), которые предоставляют пользователю информацию и элементы управления для выполнения определенных задач в интерактивной системе [18].

На рисунке 5 представлен прототип главной страницы сайта. На ней представлена актуальная статистика распространения COVID-19 по регионам России и кнопки, по нажатию которых пользователь может переходить к соответствующим разделам сайта.

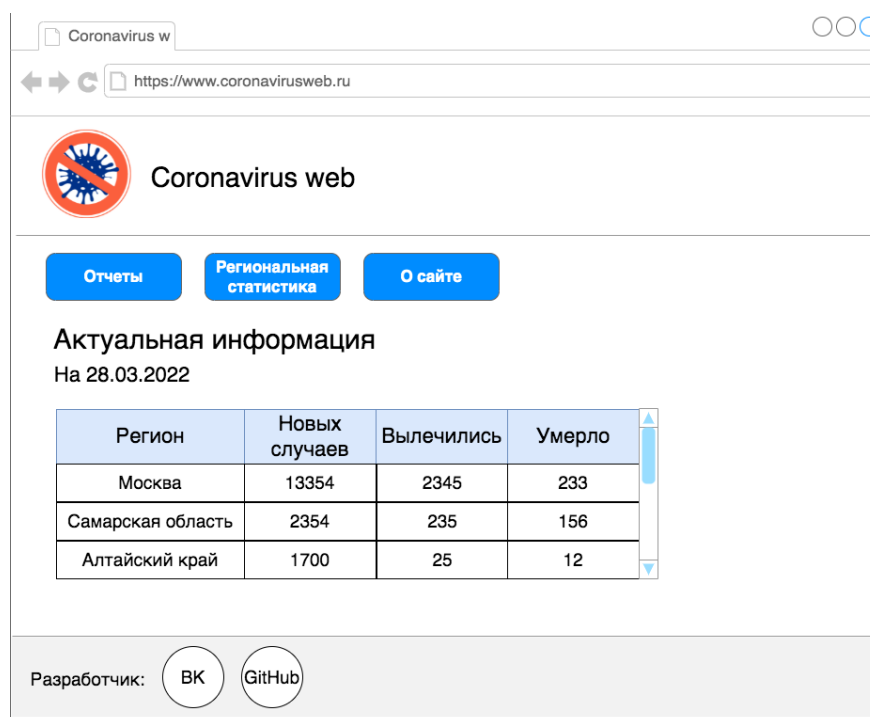


Рисунок 5 – Прототип главной страницы

На рисунке 6 представлен прототип страницы формирования отчета.

Coronavirus w

https://www.coronavirusweb.ru

 Coronavirus web На главную

Формирование отчета

Регион:

Период: с по

☒ Включить в отчет анализ мер

☒ Отправить на электронную почту

Создать отчет

Разработчик:  

Рисунок 6 – Прототип страницы формирования отчета

Здесь пользователь с помощью форм ввода может выбрать интересующий его регион, период времени, выбрать, включать ли в отчет анализ принимаемых мер, а также ввести адрес электронной почты для отправки документа. После нажатия на кнопку «Создать отчет» система сформирует в формате текстового документа отчет, начнет его скачивание на устройство пользователя, и если был введен адрес электронной почты - отправит документ на указанный адрес.

На рисунке 7 представлен прототип страницы региональной статистики. Здесь пользователь может выбрать интересующие его регион России и даты получить актуальные данные, а также скачать их по нажатию кнопки «Скачать данные».

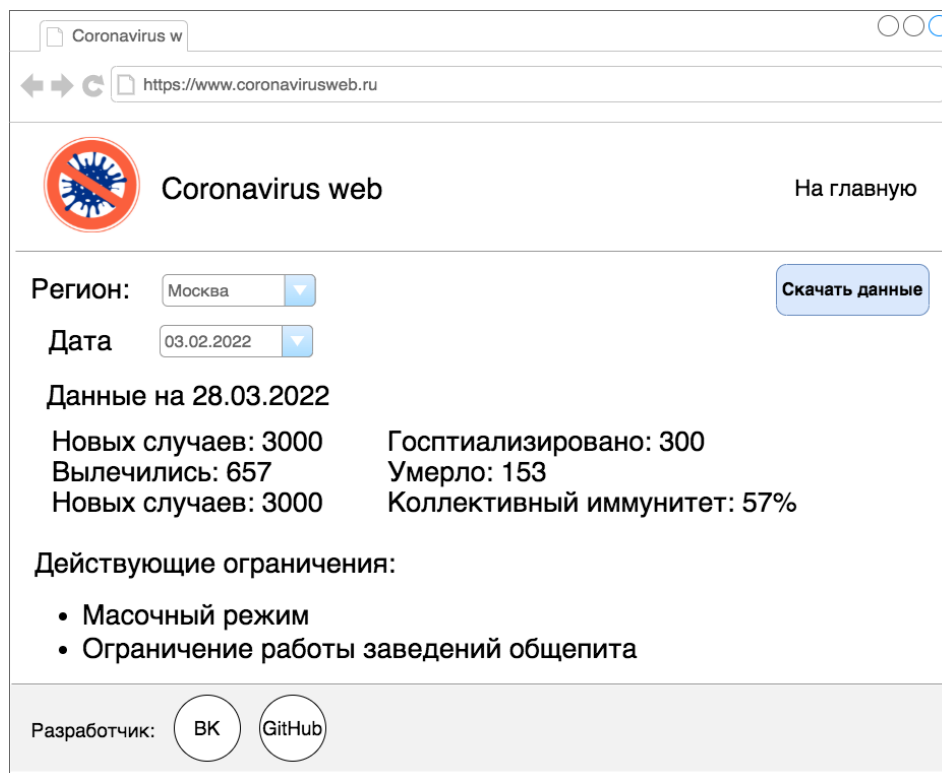


Рисунок 7 – Прототип страницы региональной статистики

На рисунке 8 представлен прототип страницы, содержащей информацию о сайте. Здесь будет представлена справочная информация и руководство пользователя.

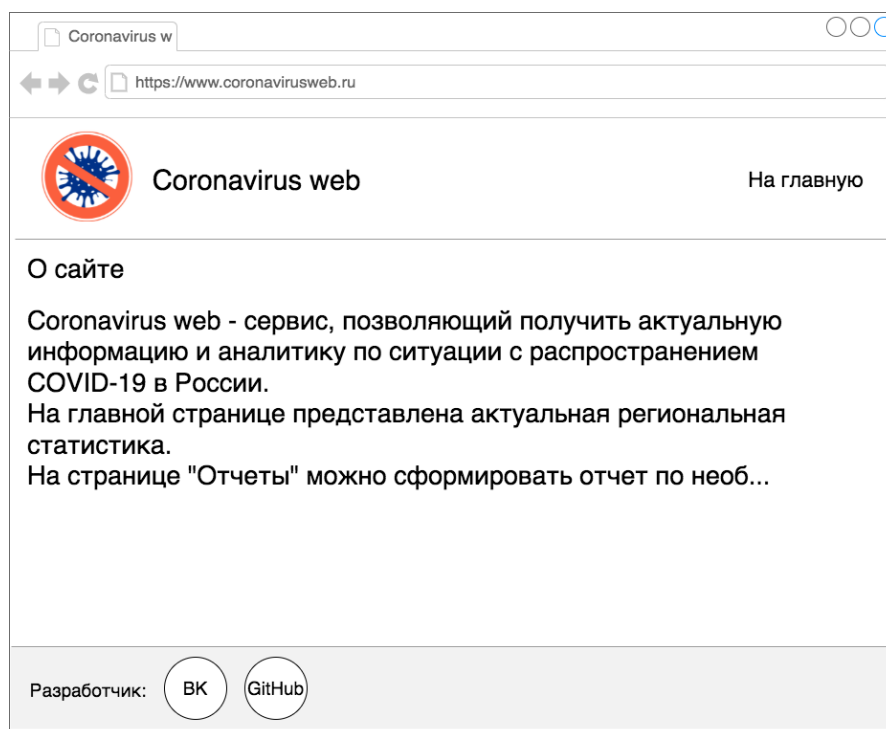


Рисунок 8 – Прототип страницы информации о сайте

2.4 Разработка информационно-логического проекта системы

2.4.1 Язык UML

Одним из инструментов разработки спецификации системы является язык UML.

Унифицированный язык моделирования (Unified Modeling Language – UML) – это стандартный инструмент для разработки «чертежей» программного обеспечения. Его можно использовать для визуализации, спецификации, конструирования и документирования артефактов программных систем. UML подходит для моделирования любых систем – от информационных систем масштаба предприятия до распределенных Web-приложений и даже встроенных систем реального времени [17].

В нотации языка UML определены следующие виды канонических диаграмм:

- вариантов использования (use case diagram);
- классов (class diagram);
- кооперации (collaboration diagram);
- последовательности (sequence diagram);
- состояний (statechart diagram);
- деятельности (activity diagram);
- компонентов (component diagram);
- развертывания (deployment diagram).

2.4.2 Диаграмма вариантов использования

Визуальное моделирование с использованием нотации UML начинается с построения модели в форме диаграммы вариантов использования (use case diagram). Данная диаграмма описывает функциональное назначение системы и является исходным концептуальным представлением в процессе ее проектирования и разработки. На ней изображаются отношения между актерами и вариантами использования.

Актер (actor) – согласованное множество ролей, которые играют внешние сущности по отношению к вариантам использования при взаимодействии с ними (это может быть любой объект, субъект или система, взаимодействующая с моделируемой бизнес-системой извне, т.е. человек, техническое устройство, программа и т.п.).

Вариант использования – внешняя спецификация последовательности действий, которые система или другая сущность могут выполнять в процессе взаимодействия с актерами (он определяет набор действий, совершаемый системой при диалоге с актером).

На рисунке 9 представлена диаграмма вариантов использования для роли пользователя «Посетитель сайта». В данной роли пользователю доступны функции просмотра актуальной статистики, статистики по регионам, информации о сайте, а также формирование отчетов.

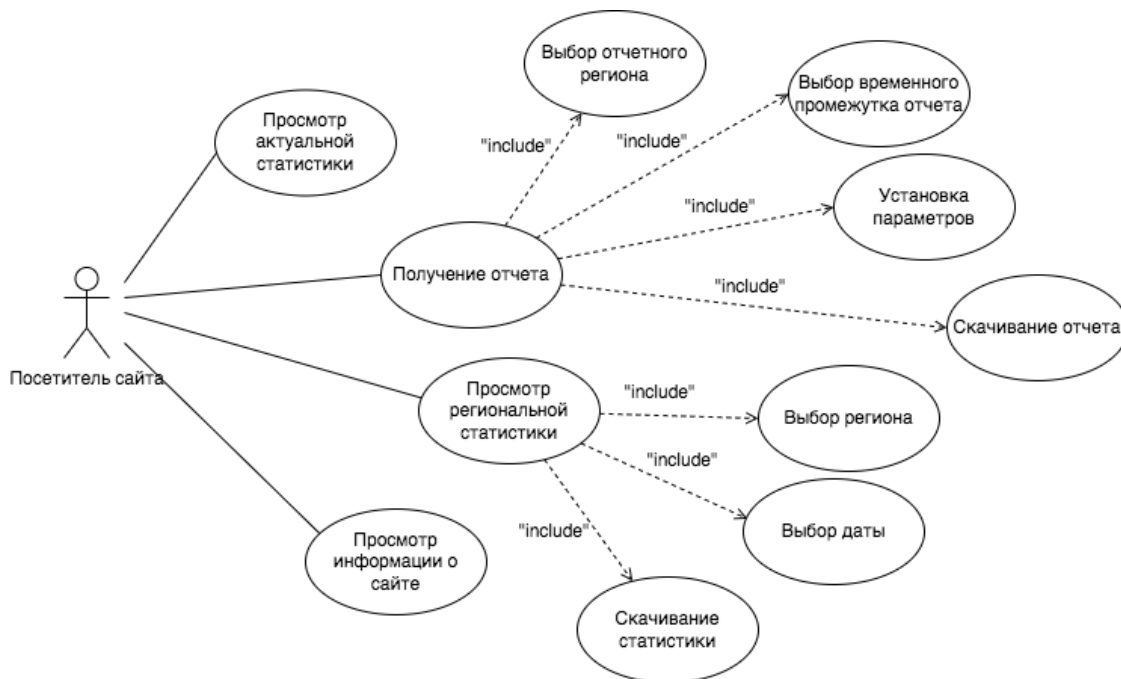


Рисунок 9 – Диаграмма вариантов использования (режим посетителя сайта)

На рисунке 10 представлена диаграмма вариантов использования для роли пользователя «Администратор». Администратору доступны все функции посетителя, а также доступ к базе данных: просмотр и редактирование записей в базе данных, а также запуск функции получения актуальной информации.

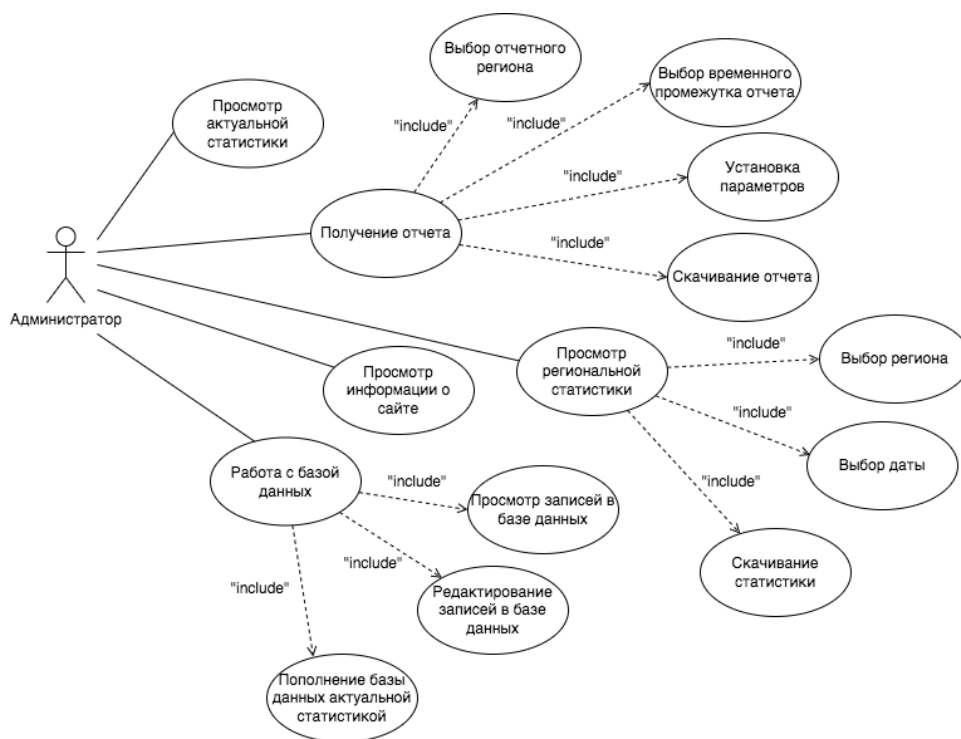


Рисунок 10 – Диаграмма вариантов использования (режим администратора)

2.4.3 Диаграмма классов

Диаграммы классов – это наиболее часто используемый тип диаграмм, которые создаются при моделировании объектно-ориентированных систем, они показывают набор классов, интерфейсов и коопераций, а также их связи. На практике диаграммы классов применяют для моделирования статического представления системы, они служат основой для целой группы взаимосвязанных диаграмм – диаграмм компонентов и диаграмм размещения [17].

На данном этапе для всех информационных объектов, выделенных в системе, разрабатываются классы с указанием полей, методов и свойств, которые регулируют процессы обработки данных (потoki данных заданной структуры) и/или структуры данных.

На рисунке 11 представлена диаграмма классов системы. В таблице 2 дано описание классов

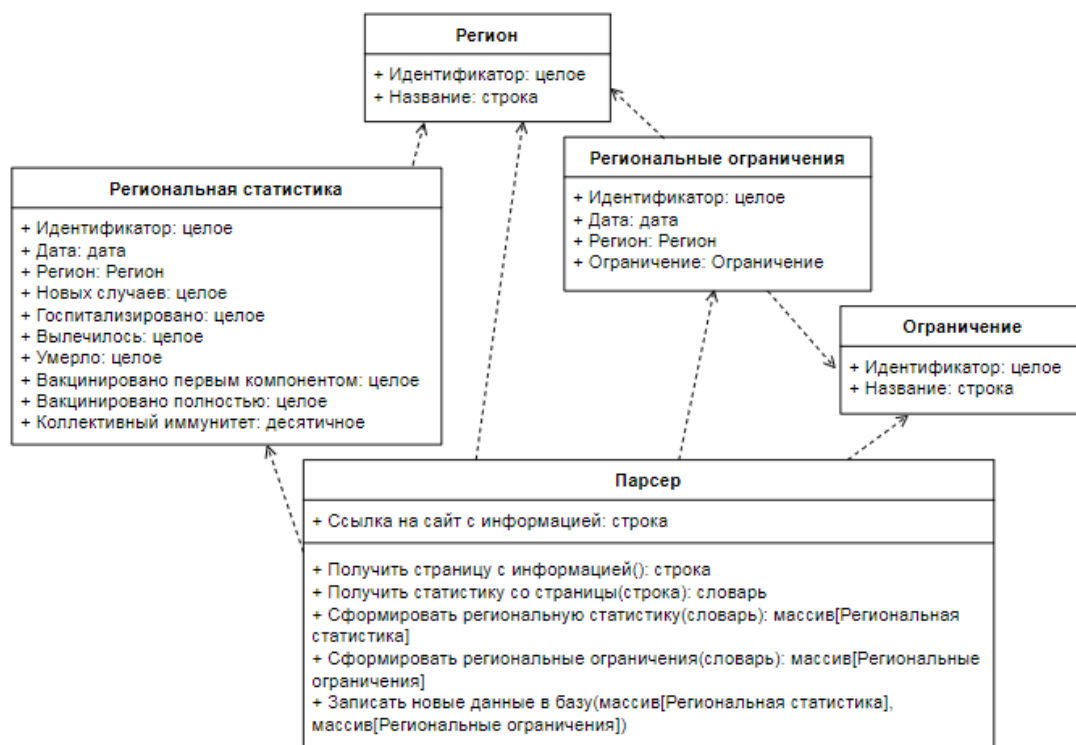


Рисунок 11 – Диаграмма классов системы

Таблица 2 – Описание классов системы

Название класса	Назначение
Регион	Хранение информации о регионах
Региональная статистика	Хранение информации о региональной статистике
Ограничение	Хранение информации об ограничениях
Региональные ограничения	Хранение информации о региональных ограничениях
Парсер	Обновление информации в базе данных

2.4.4 Сценарий

Сценарий (scenario) – определенная последовательность действий, которая описывает действия актеров и поведение моделируемой системы в форме обычного текста [17].

Рассмотрим сценарий для варианта использования «Получение отчета».

Краткое описание: дает пользователю возможность сформировать аналитический отчет по выбранным данным.

Актант: пользователь.

Предусловия: компьютер пользователя включен. На компьютере открыт браузер. Пользователь зашел на главную страницу сайта /stats/. Пользователь нажал на кнопку «Создать отчет» на главной странице сайта.

Основной поток событий.

1 Система открывает в браузере пользователя страницу создания отчета (см. рисунок 6), содержащую выпадающие списки для выбора региона и временного периода, чекбоксы для выбора включения в отчет аналитики и отправки отчета на электронную почту, текстовое поле для ввода адреса электронной почты и кнопку «Создать отчет». Поле ввода почты заблокировано, кнопка «Создать отчет» недоступна к нажатию.

2 Пользователь в контекстном меню выбирает отчетный регион

A1: Пользователь закрывает вкладку.

3 Пользователь в контекстном меню выбирает дату начала отчетного периода.

4 Пользователь в контекстном меню выбирает дату конца отчетного периода.

5 Система проверяет корректность временного периода.

A2: Дата конца выставлена раньше даты начала отчетного периода.

6 Пользователь отмечает чекбокс «Включить в отчет анализ мер».

A3: Пользователь не отмечает чекбокс «Включить в отчет анализ мер».

7 Пользователь отмечает чекбокс «Отправить отчет на электронную почту».

А4: Пользователь не отмечает чекбокс «Отправить отчет на электронную почту».

8 Система делает поле ввода электронной почты доступным для ввода.

9 Пользователь вводит в поле адрес электронной почты.

10 Система проверяет корректность ввода электронной почты.

11 Система делает кнопку «Создать отчет» доступной для нажатия.

А5: Электронная почта введена некорректно.

12 Пользователь нажимает кнопку «Создать отчет».

13 Система получает введенные данные из элементов экрана.

14 Система создает текстовый файл.

15 Система добавляет в отчет статистику распространения коронавируса в выбранном регионе по выбранным датам.

16 Система добавляет в отчет анализ мер противодействия.

А6: Чекбокс «Включить в отчет анализ мер» не был отмечен.

17 Система сохраняет внесенные в файл изменения.

18 Система загружает текстовый файл с отчетом на устройство пользователя.

19 Система отправляет текстовый файл с отчетом на указанную электронную почту.

А7: Чекбокс «Отправить отчет на электронную почту» не был отмечен.

20 Пользователю выводится всплывающее уведомление «Отчет успешно сформирован». Вариант использования завершился успешно.

Альтернативы.

А1: Пользователь закрывает вкладку.

А1.1: Взаимодействие с системой завершено. Вариант использования завершается.

А2: Дата конца выставлена раньше даты начала отчетного периода.

A2.1: Система выводит на страницу текст «Некорректное значение промежутка дат».

A2.2: Переход к п. 3 основного потока или п. 4 основного потока.

A3: Пользователь не отмечает чекбокс «Включить в отчет анализ мер».

A3.1: Переход к п. 7 основного потока.

A4: Пользователь не отмечает чекбокс «Отправить отчет на электронную почту».

A4.1: Переход к п. 12 основного потока.

A5: Электронная почта введена некорректно.

A5.1: Система выводит на страницу текст «Некорректное значение адреса электронной почты».

A5.2: Переход к п. 7 основного потока или п. 9 основного потока.

A6: Чекбокс «Включить в отчет анализ мер» не был отмечен.

A6.1: Переход к п. 17 основного потока.

A7: Чекбокс «Отправить отчет на электронную почту» не был отмечен.

A7.1: Переход к п. 20 основного потока.

Постусловия. При успешном завершении на устройство пользователя скачан текстовый документ, содержащий данные по распространению коронавируса в выбранном регионе за выбранный промежуток времени. Если был выбран анализ мер - текстовый документ также содержит анализ мер. Если была введена электронная почта - документ также был отправлен на электронную почту.

2.4.5 Диаграмма состояний

Для моделирования поведения на логическом уровне в языке UML могут использоваться сразу несколько канонических диаграмм: состояний, деятельности, последовательности и кооперации, каждая из которых фиксирует внимание на отдельном аспекте функционирования системы. В отличие от других диаграмм диаграмма состояний описывает процесс

изменения состояний только одного экземпляра определенного класса, т. е. моделирует все возможные изменения в состоянии конкретного объекта.

Главное предназначение этой диаграммы – описать возможные последовательности состояний и переходов, которые в совокупности характеризуют поведение элемента модели в течение его жизненного цикла. Диаграмма состояний представляет динамическое поведение сущностей, на основе спецификации их реакции на восприятие некоторых событий [17].

На рисунке 12 представлена диаграмма состояний системы. При открытии стартовой страницы сайта система отображает главную страницу, подключается к базе данных, загружает последнюю информацию и отображает ее в таблице. Пользователь может переходить к другим разделам сайта с помощью кнопок.

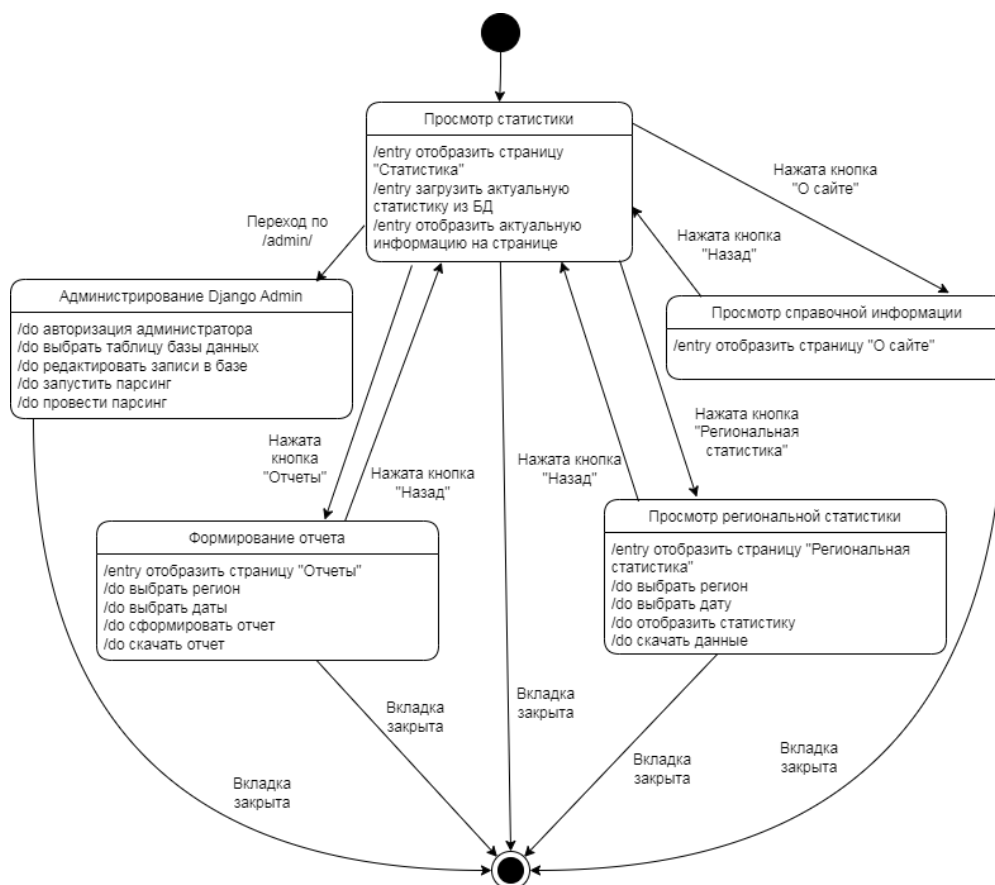


Рисунок 12 – Диаграмма состояний системы

При нажатии кнопки «Региональная статистика» система открывает страницу региональной статистики. Пользователь может выбрать регион и дату с помощью элементов страницы. После этого система загружает из базы

данных и отображает необходимую статистику. При нажатии на кнопку «Скачать данные» отображенная информация будет скачана на устройство пользователя. При нажатии кнопки «Назад» система возвращает пользователя к стартовой странице.

При нажатии кнопки «О сайте» система открывает страницу со справочной информацией о сайте. На ней пользователь может ознакомиться с руководством пользователя и другой справочной информацией.

При нажатии кнопки «Отчеты» система открывает страницу формирования отчета. На ней пользователь может выбрать регион и даты, а также настроить, включать ли в отчет анализ мер и отправлять ли отчет на электронную почту. Адрес электронной почты вписывается в поле ввода. При нажатии кнопки «Создать отчет» система формирует отчет по указанной информации и загружает его на устройство пользователя, а также, если был указан адрес почты, отправляет на него отчет.

При переходе по ссылке к разделу сайта /admin/ откроется интерфейс Django Admin. Система откроет страницу авторизации администратора. При введении правильных логина и пароля система откроет панель управления. На ней администратор может просматривать и редактировать записи различных таблиц базы данных сайта. Также в панели управления будет доступна кнопка «Провести парсинг». По нажатию данной кнопки система запустит процесс парсинга актуальной информации и записи их в базу данных.

2.4.6 Диаграмма деятельности

Диаграмма деятельности (активностей – Activity Diagram) применяется для моделирования динамических аспектов поведения системы. Это частный случай диаграммы состояний, она позволяет реализовать особенности процедурного и синхронного управления, обусловленного завершением внутренних действий и деятельности [17].

На рисунке 13 представлена диаграмма деятельности системы при варианте использования «Получение отчета».

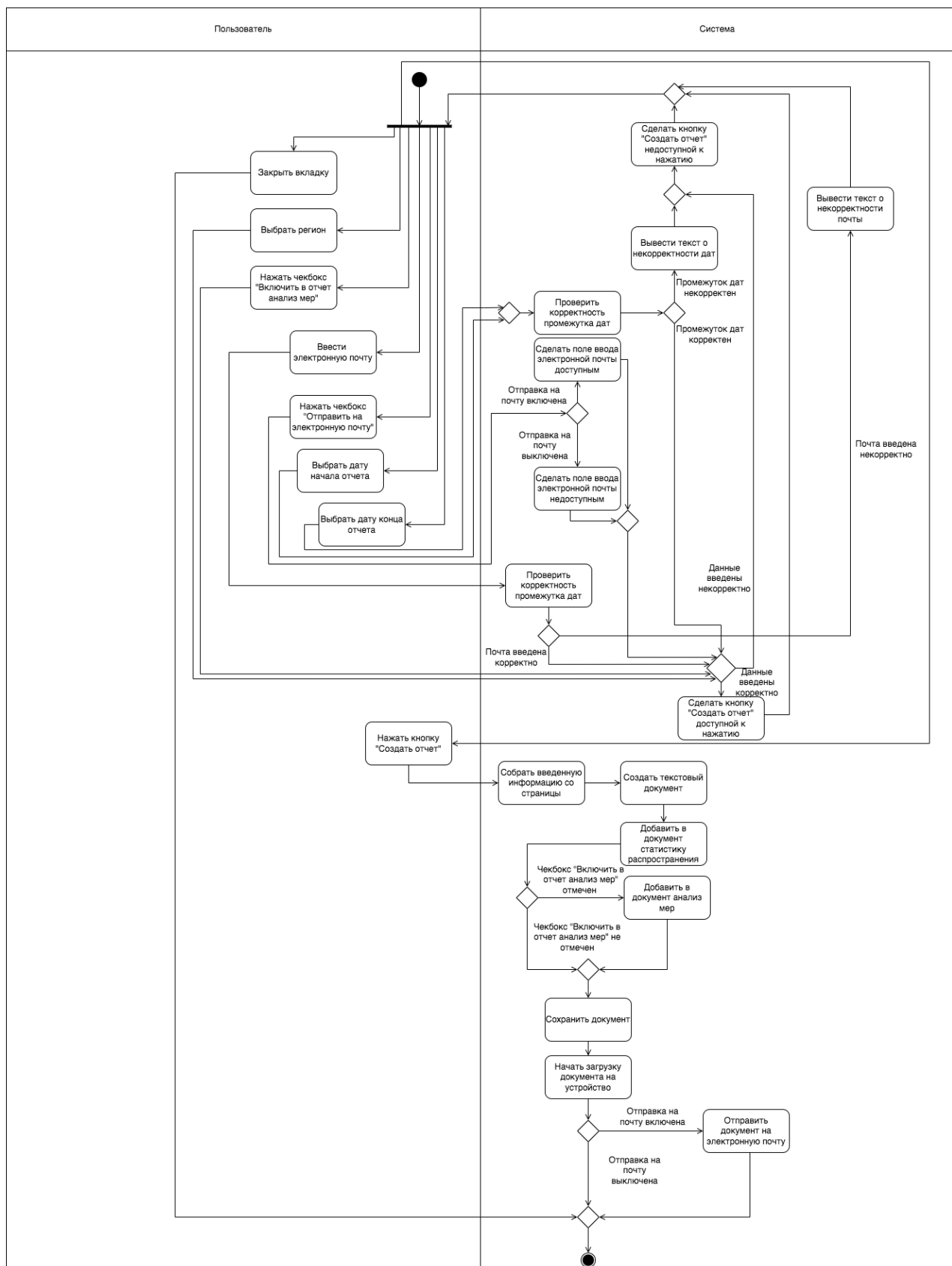


Рисунок 13 – Диаграмма деятельности системы при варианте использования «Получение отчета»

Пользователь может выбрать регион, даты начала и конца, включение анализа мер, отправку на электронную почту, а также ввести свой адрес электронной почты. После каждого выбора пользователя система проводит проверку корректности, и при корректных введенных данных делает доступной кнопку «Создать отчет». По нажатию пользователем кнопки «Создать отчет» система формирует текстовый файл отчета, начинает процесс его скачивания и отправки на почту, если был выбран соответствующий пункт.

2.4.7 Диаграмма последовательности

Диаграмма последовательности – диаграмма, на которой для некоторого набора объектов на единой временной оси показан жизненный цикл какого-либо определённого объекта (создание, деятельность и уничтожение некой сущности) и взаимодействие актёров в рамках какого-либо определённого прецедента (отправка запросов и получение ответов). Используется в языке UML [19].

На рисунках 14-16 представлена диаграмма последовательности для варианта использования «Получение отчета». Диаграммы построены на основании сценария, приведенного в п.2.4.4.

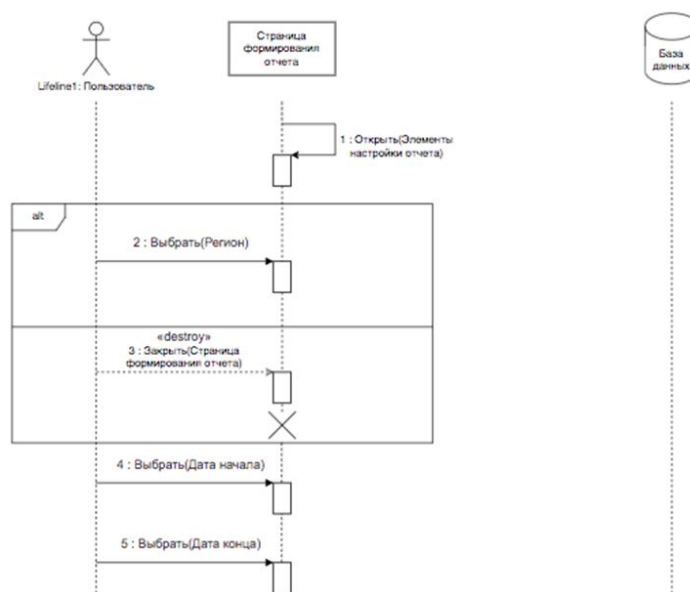


Рисунок 14 – Диаграмма последовательности варианта использования «Получение отчета» (часть 1)

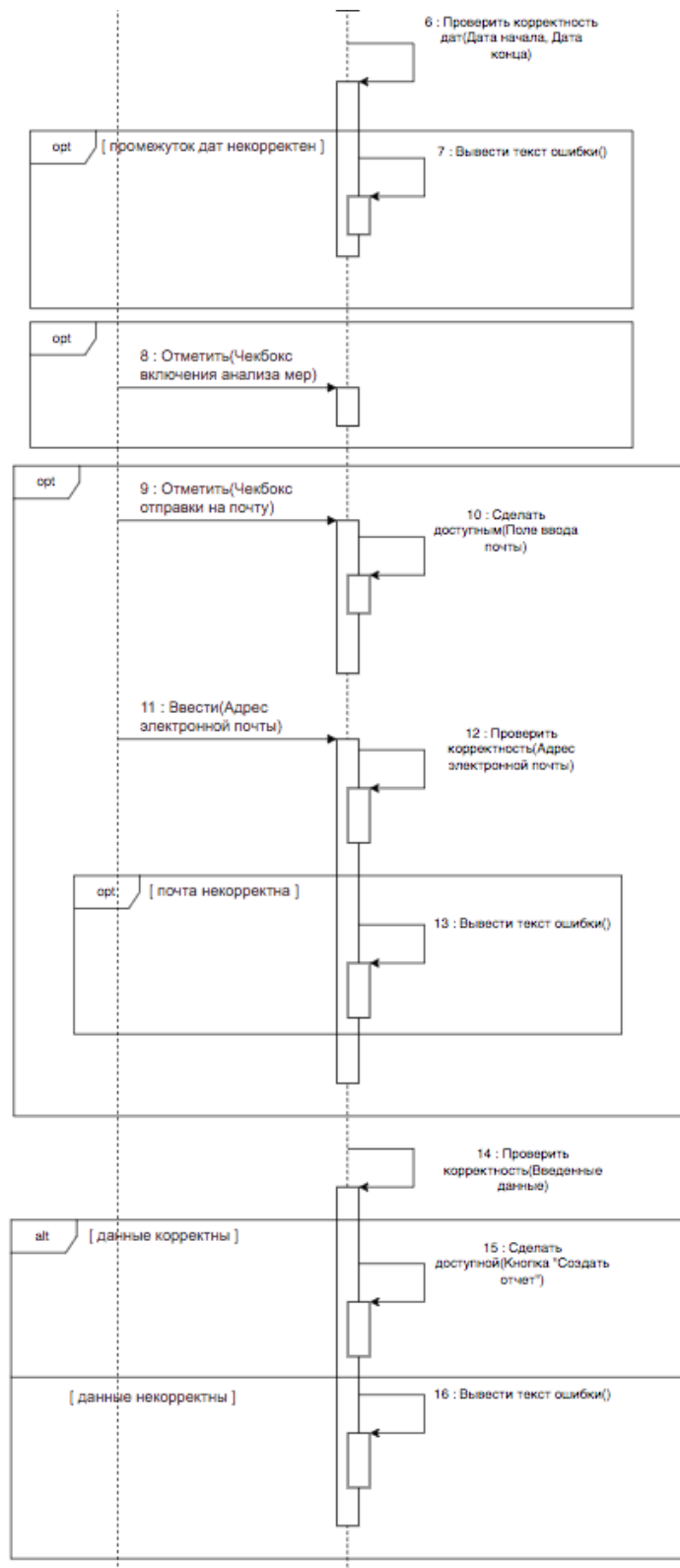


Рисунок 15 – Диаграмма последовательности варианта использования «Получение отчета» (часть 2)

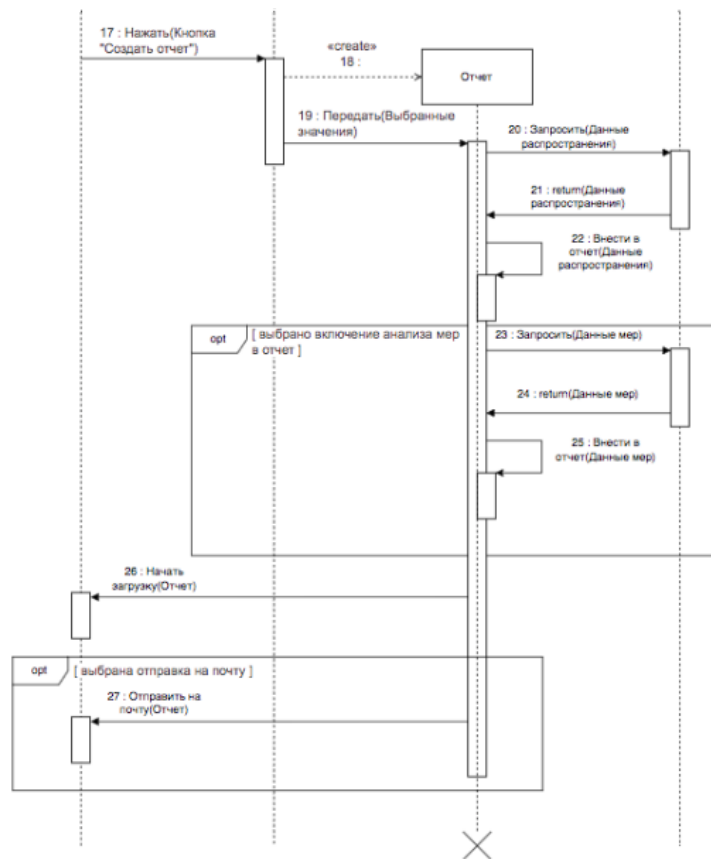


Рисунок 16 – Диаграмма последовательности варианта использования «Получение отчета» (часть 3)

2.4.8 Логическая модель данных системы

Логическая информационная модель – модель данных, в которой учитывается способ логического хранения данных в памяти ЭВМ. При построении модели базы данных (БД) используются следующие понятия.

Сущность – объект предметной области, который можно отличить от других понятий по некоторым признакам. Сущность состоит из множества своих экземпляров. Каждая сущность обладает свойствами – атрибутами [20].

Атрибут – определенное свойство сущности. Именно набор атрибутов, в общем случае уникальный для каждой сущности, позволяет выделить ее среди других объектов и назвать уникальным именем.

Атрибут или набор атрибутов, используемый для идентификации экземпляра сущности, называется ключом сущности. В случае если для идентификации экземпляра используется один атрибут, ключ называется

простым; в противном случае ключ составной. Каждый экземпляр сущности однозначно определяется ключом [20].

Логическая модель базы данных разрабатываемой системы приведена на рисунке 17.

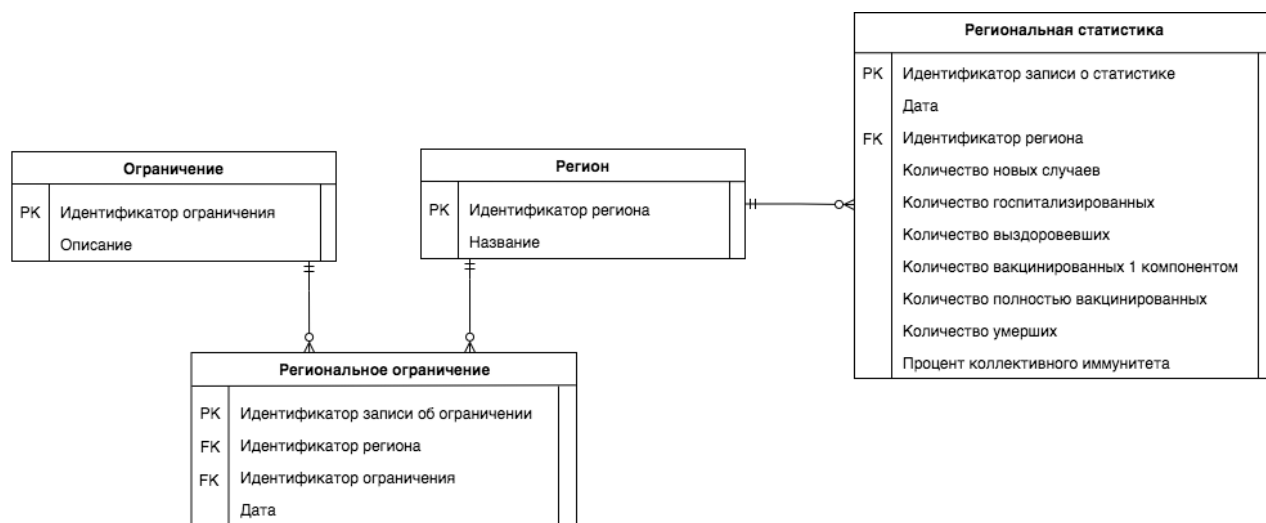


Рисунок 17 – Логическая модель базы данных системы

Описание объектов рассматриваемой предметной области, которые хранятся в базе данных, приведено в таблицах 3-6.

Таблица 3 – Сущность «Ограничение»

Идентификатор	Тип данных	Описание
Идентификатор ограничения	Целый	Уникальный идентификатор ограничения
Описание	Текстовый	Описание ограничения

Таблица 4 – Сущность «Регион»

Идентификатор	Тип данных	Описание
Идентификатор региона	Целый	Уникальный идентификатор региона
Название	Символьный[100]	Название региона

Таблица 5 – Сущность «Региональное ограничение»

Идентификатор	Тип данных	Описание
Идентификатор записи об ограничении	Целый	Уникальный идентификатор записи о региональном ограничении
Идентификатор региона	Целый	Идентификатор региона, в котором введено ограничение
Идентификатор ограничения	Целый	Идентификатор введенного ограничения
Дата	Дата	Дата действия ограничения

Таблица 6 – Сущность «Региональная статистика»

Идентификатор	Тип данных	Описание
Идентификатор записи о статистике	Целый	Уникальный идентификатор записи о статистике распространения
Дата	Дата	Дата собранной статистики
Идентификатор региона	Целый	Идентификатор региона, в котором введено ограничение
Количество новых случаев	Целый	Количество новых случаев заражения в данную дату
Количество госпитализированных	Целый	Количество госпитализированных в данную дату
Количество выздоровевших	Целый	Количество выздоровевших в данную дату
Количество вакцинированных 1 компонентом	Целый	Суммарное количество вакцинированных первым компонентом
Количество полностью вакцинированных	Целый	Суммарное количество полностью вакцинированных
Количество умерших	Целый	Количество умерших в данную дату
Процент коллективного иммунитета	Десятичный	Процент коллективного иммунитета в регионе в данную дату

2.5 Выбор и обоснование комплекса программных средств

2.5.1 Выбор языка программирования

Система разрабатывается на языке программирования Python с использованием фреймворка Django.

Python – высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью, ориентированный на повышение производительности разработчика, читаемости кода и его качества, а также на обеспечение переносимости написанных на нём программ [21].

Django – высокоуровневый веб фреймворк для Python, способствующий быстрой разработке и чистому дизайну создаваемых веб-приложений. Данный фреймворк является бесплатным, позволяет легко и быстро развивать систему и расширять функционал, а также содержит удобную встроенную панель администрирования.

2.5.2 Выбор среды программирования

Программа была реализована в среде разработки PyCharm. PyCharm – интегрированная среда разработки для языка программирования Python. Предоставляет средства для анализа кода, графический отладчик, инструмент для запуска юнит-тестов и поддерживает веб-разработку на Django. PyCharm разработана компанией JetBrains на основе IntelliJ IDEA [22].

PyCharm имеет следующие возможности:

- отладка кода при помощи PyDev;
- рефакторинг кода;
- поддержка Git, SVN, Mercurial и других систем контроля версиями;
- автодополнение кода.

2.5.3 Выбор СУБД

В качестве СУБД был выбран PostgreSQL. PostgreSQL — это система объектно-реляционных баз данных с открытым исходным кодом, которая

использует и расширяет язык SQL в сочетании со многими функциями, позволяющими безопасно хранить и масштабировать самые сложные рабочие нагрузки данных [23].

PostgreSQL обладает следующими достоинствами:

- высокая программируемость;
- оптимизация;
- поддержка фреймворков Django баз данных на PostgreSQL.

3 Реализация системы

3.1 Разработка и описание интерфейса пользователя

При запуске сайта открывается главная страница (см. рисунок 18).



Рисунок 18 – Главная страница сайта

На данной странице представлена таблица с актуальной информацией по распространению коронавируса в регионах России, а также кнопка, по нажатию которой пользователь может скачать актуальные данные в формате JSON. В футере сайта расположены ссылки на страницы разработчика в ВК и на GitHub. Также в верхней части страницы расположены кнопки для перехода к разделам сайта.

При нажатии пользователем на кнопку «Сформировать отчет» на главной странице система переводит пользователя на страницу формирования аналитических отчетов. Внешний вид страницы представлен на рисунке 19.

Сформировать отчет:

Регион

Дата начала

Дата конца

Введите обе даты

☐ Включить в отчет анализ мер

☒ Отправить отчет на электронную почту

Адрес

Разработчик:

Рисунок 19 – Страница формирования отчетов

На данной странице пользователь с помощью полей ввода может выбрать регион, даты начала и конца отчетного периода, включить в отчет анализ мер противодействия пандемии, а также включить отправку отчета на выбранный адрес электронной почты.

Система отслеживает корректность введения данных. При корректном введении всех данных становится доступна для нажатия кнопка «Создать отчет». При нажатии данной кнопки формируется аналитический отчет по заданным параметрам, скачивается на устройство пользователя и, если была выбрана опция отправки на почту, отправляется на указанную в поле ввода почту.

Нажатие кнопки «Назад» возвращает пользователя на главную страницу сайта. Футер страницы аналогичен находящемуся на главной странице.

При нажатии пользователем на кнопку «Региональная статистика» на главной странице сайта система переводит пользователя на страницу просмотра региональной статистики. Внешний вид страницы представлен на рисунке 20.


Статистика коронавируса в России

Назад

Историческая статистика по регионам

Регион

Республика Адыгея

Дата начала

04.05.2022

Получить данные

Регион: Республика Адыгея

Дата: 4 мая 2022 г.

Новых случаев: 11

Госпитализаций: 3

Смертей: 0

Вакцинировано 1 компонентом: 218248

Вакцинировано полностью: 211235

Коллективный иммунитет: 35,5

Действующие ограничения:

- Обязательная вакцинация введена для госслужащих и сотрудников подведомственных учреждений, в т.ч. организаций здравоохранения, образования, социальной защиты
- Введен масочный режим
- Введен масочный режим в общественном транспорте
- Введен удаленный режим работы для людей с хроническими заболеваниями, граждан 65+ и беременных

Рисунок 20 – Страница региональной статистики

На данной странице пользователь может выбрать регион и дату. Нажатие кнопки «Получить данные» выдает в нижнем блоке соответствующую информацию. Нажатие кнопки «Назад» возвращает пользователя на главную страницу сайта.

При нажатии на кнопку «Информация о сайте» на главной странице переводит пользователя на страницу просмотра региональной статистики (см. рисунок 21). Здесь представлена справочная информация, а также руководство пользователя.


Статистика коронавируса в России

Назад

Работа с сайтом

При входе на сайт открывается главная страница (см. рисунок 1). С помощью кнопок «Сформировать отчет», «Региональная статистика» и «Информация о сайте» пользователь может переходить к разделам сайта. Также на странице представлена актуальная информация о распространении COVID-19 в виде таблицы. Нажатие кнопки «Скачать данные» скачивает на компьютер пользователя JSON-файл, содержащий данные этой таблицы. Кнопки в футере страницы переводят к страницам разработчика в VK и GitHub.

Сформировать отчет

Региональная статистика

Информация о сайте

Актуальная информация

на 18 мая 2022 г.

Регион	Новых случаев	Госпитализировано	Выздоровело	Умерло
Россия	4732	2395	6134	106
Москва	380	117	359	17
Санкт-Петербург	361	105	748	8
Московская область	187	82	188	0
Республика Татарстан	180	86	226	1
Свердловская область	161	60	194	1
Воронежская область	154	28	156	1
Валгоградская область	144	54	129	0
Саратовская область	129	48	142	1
Челябинская область	127	65	135	6

Скачать данные

Рисунок 21 – Страница информации о сайте

3.2 Диаграммы реализации

Диаграммы реализации – это обобщающее название для диаграмм компонентов и диаграмм размещения [24].

Название объясняется тем, что данные диаграммы приобретают особую важность на позднейших фазах разработки – на фазах реализации и перехода. На ранних фазах разработки эти диаграммы либо вообще не используются, либо имеют самый общий, не детализированный вид.

3.2.1 Диаграмма развертывания

Диаграмма развертывания предназначена для визуализации элементов и компонентов программы, существующих лишь на этапе ее исполнения [25]. При этом представляются только компоненты-экземпляры программы – исполняемые файлы или динамические библиотеки. Компоненты, которые не используются на этапе исполнения, на диаграмме не показываются.

На рисунке 22 представлена диаграмма развертывания системы. Сервер содержит операционную систему, СУБД PostgreSQL и сервер WSGI, на котором работает система. Персональный компьютер клиента должен содержать операционную систему и браузер.

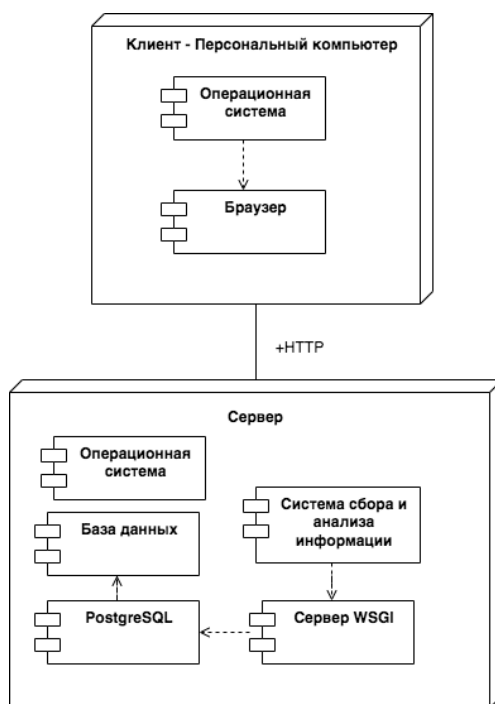


Рисунок 22 – Диаграмма развертывания системы

3.2.2 Диаграмма компонентов

Диаграммы компонентов используются для визуализации организации компонентов системы и зависимостей между ними. Они позволяют получить высокоуровневое представление о компонентах системы [26].

Компонентами могут быть программные компоненты, такие как база данных или пользовательский интерфейс; или аппаратные компоненты, такие как схема, микросхема или устройство; или бизнес-подразделение, такое как поставщик, платежная ведомость или доставка.

Компонентные диаграммы:

- используются в компонентно-ориентированных разработках для описания систем с сервис-ориентированной архитектурой;
- показать структуру самого кода;
- могут использоваться для фокусировки на отношениях между компонентами, скрывая при этом детализацию спецификации;
- помогают в информировании и разъяснении функций создаваемой системы заинтересованным сторонам.

На рисунке 23 приведена диаграмма компонентов, их описание приведено в таблице 7.

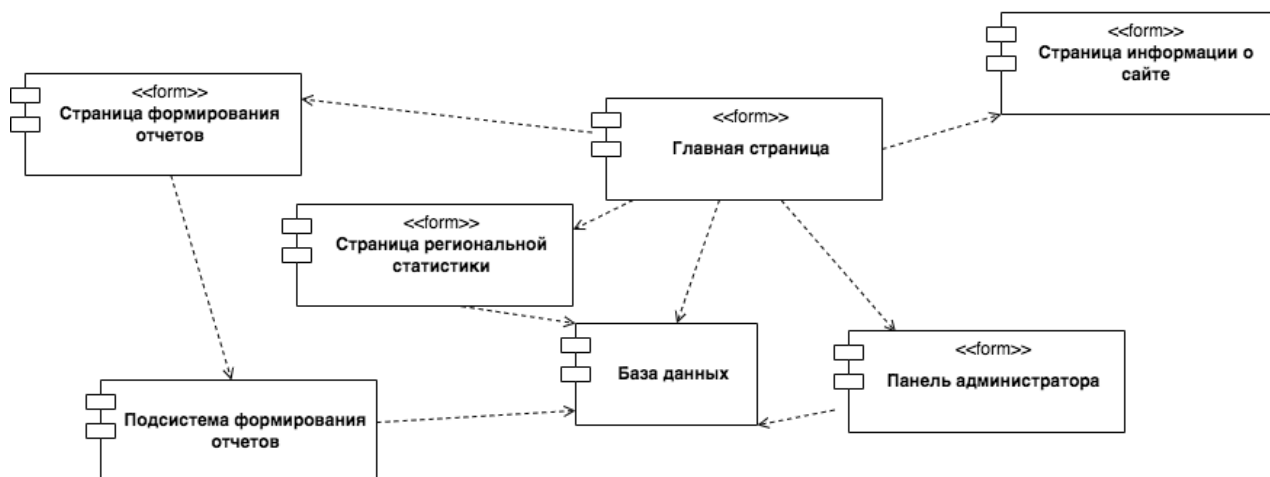


Рисунок 23 – Диаграмма компонентов системы

Таблица 7 – Описание компонентов системы

Название компонента	Назначение компонента
Главная страница	Предназначена для отображения и скачивания актуальной информации и навигации пользователя по разделам сайта
Страница информации о сайте	Предназначена для вывода справочной информации о системе и руководства пользователя системы
Страница формирования отчетов	Предназначена для ввода параметров и получения отчетов
Страница региональной статистики	Предназначена для просмотра исторических региональных данных по распространению коронавируса
Подсистема формирования отчетов	Предназначена для формирования аналитических отчетов в формате документа Word
Панель администратора	Предназначена для просмотра и редактирования базы данных, а также запуска процесса сбора актуальной информации
База данных	Предназначена для хранения данных системы

3.2.3 Диаграмма классов системы

На рисунке 24 приведена диаграмма классов системы (этап реализации).

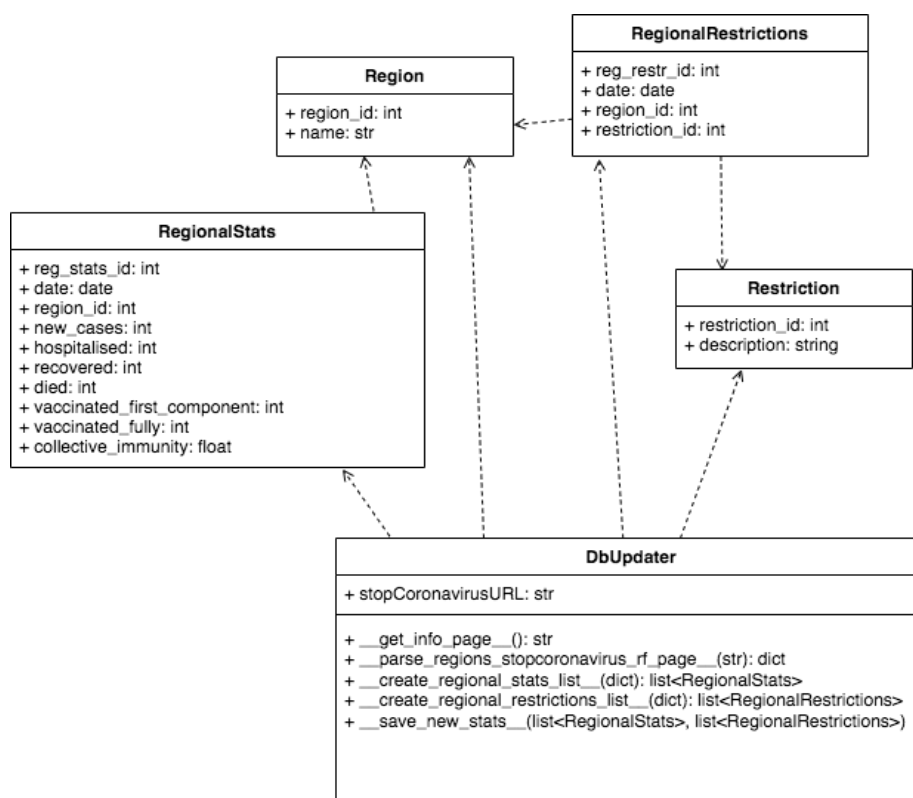


Рисунок 24 – Диаграмма классов системы

3.2.4 Физическая схема данных

Физическое проектирование – создание схемы базы данных для конкретной СУБД. Специфика конкретной СУБД может включать в себя ограничения на именование объектов базы данных, ограничения на поддерживаемые типы данных и т.п. Кроме того, специфика конкретной СУБД при физическом проектировании включает выбор решений, связанных с физической средой хранения данных (выбор методов управления дисковой памятью, разделение БД по файлам и устройствам, методов доступа к данным), создание индексов и т.д. [27].

В таблице 8 приведено описание соответствия сущностей логической модели и таблиц физической модели.

Таблица 8 – Соответствие сущностей логической и физической моделей

Сущность логической модели	Сущность физической модели
Регион	Region
Ограничение	Restriction
Региональная статистика	RegionalStats
Региональное ограничение	RegionalRestrictions

На рисунке 25 представлена физическая модель данных системы.

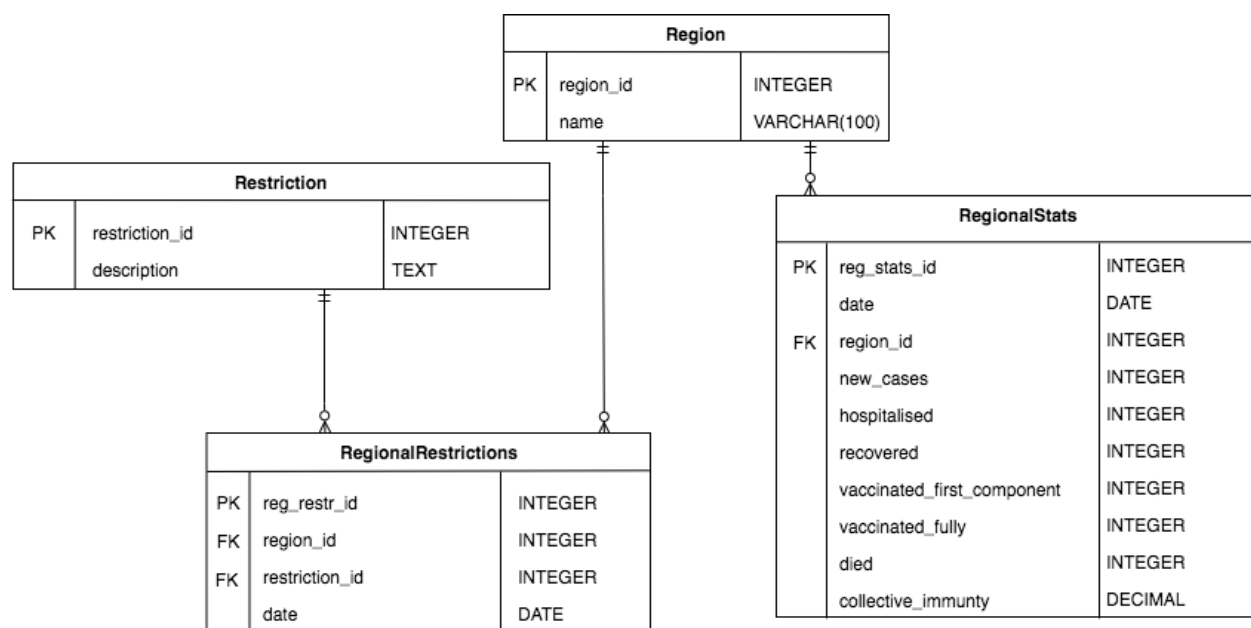


Рисунок 25 – Физическая модель данных системы

В таблицах 9-12 приведено описание сущностей БД. Первичные ключи выделены жирным шрифтом, а внешние – курсивом.

Таблица 9 – Сущность «Restriction»

Имя поля	Имя атрибута	Тип	Размер (байт)
restriction_id	Идентификатор ограничения	int	4
description	Описание	text	65535
Размер записи			65539

Таблица 10 – Сущность «Region»

Имя поля	Имя атрибута	Тип	Размер (байт)
region_id	Идентификатор региона	int	4
name	Название	varchar(100)	100
Размер записи			104

Таблица 11 – Сущность «RegionalRestrictions»

Имя поля	Имя атрибута	Тип	Размер (байт)
reg_restr_id	Идентификатор записи об ограничении	int	4
<i>region_id</i>	<i>Идентификатор региона</i>	<i>int</i>	<i>4</i>
<i>restriction_id</i>	<i>Идентификатор ограничения</i>	<i>int</i>	<i>4</i>
date	Дата	date	4
Размер записи			16

Таблица 12 – Сущность «RegionalStats»

Имя поля	Имя атрибута	Тип	Размер (байт)
1	2	3	4
reg_stats_id	Идентификатор записи о статистике	int	4
date	Дата	date	4
<i>region_id</i>	<i>Идентификатор региона</i>	<i>int</i>	<i>4</i>
new_cases	Количество новых случаев	int	4
hospitalised	Количество госпитализированных	int	4
recovered	Количество выздоровевших	int	4

Продолжение таблицы 12

1	2	3	4
vaccinated_first_component	Количество вакцинированных первым компонентом	int	4
vaccinated_fully	Количество полностью вакцинированных	int	4
died	Количество умерших	int	4
collective_immunity	Процент коллективного иммунитета	decimal	2
Размер записи			38

3.3 Выбор и обоснование комплекса технических средств

3.3.1 Расчет объема занимаемой памяти

Для расчета необходимого объема свободной внешней памяти сервера, необходимой для функционирования системы, воспользуемся следующей формулой:

$$V_{\text{ЖД}} = V_{\text{ОС}} + V_{\text{ПР}} + V_{\text{СПО}} + V_{\text{БД}} + V_{\text{справки}}$$

где $V_{\text{ОС}}$ – объем памяти, занимаемый операционной системой (операционная система Windows 10, $V_{\text{ОС}} = 5.5$ Гб);

$V_{\text{ПР}}$ – объем памяти, занимаемый непосредственно файлами приложения ($V_{\text{ПР}} = 13$ Мб);

$V_{\text{СПО}}$ – объем памяти, занимаемый сопутствующим программным обеспечением (фреймворк Django, библиотеки Beautiful Soup, python-docx, Matplotlib, Pillow, Psycorg 2; дадим оценку сверху $V_{\text{СПО}}$ в 1 Гб);

$V_{\text{БД}}$ – объем памяти, занимаемый базой данных (всеми таблицами) при ее максимальном заполнении. Дадим оценку сверху $V_{\text{БД}}$ в 10 Гб

Таким образом, суммарный объем внешней памяти составит:

$$V_{\text{ЖД}} = 5.5 \text{ Гб} + 13 \text{ Мб} + 1 \text{ Гб} + 10 \text{ Гб} \sim 16.5 \text{ Гб}.$$

Расчет объема ОЗУ

Для расчета необходимого объема ОЗУ воспользуемся следующей формулой:

$$V_{\text{ОЗУ}} = V_{\text{ОС}} + V_{\text{ПР}} + V_{\text{БД}},$$

где $V_{\text{ОС}}$ – ОЗУ, занимаемое операционной системой (1 Гб);

$V_{\text{ПР}}$ – ОЗУ, которое займет само приложение (не превысит 300 Мб);

$V_{\text{БД}}$ – объем данных из базы, который может быть одновременно загружен в оперативную память (дадим ему оценку сверху в 10 Мб).

Суммарные объемы ОЗУ составит:

$$V_{\text{ОЗУ}} = 1 \text{ Гб} + 300 \text{ Мб} + 10 \text{ Мб} \sim 1.3 \text{ Гб}.$$

Таким образом, 1.3 Гб оперативной памяти можно считать минимально необходимым для функционирования сервера.

Произведем аналогичные оценки для компьютера-клиента:

Внешняя память:

$V_{\text{ОС}}$ – операционная система Windows 10, $V_{\text{ОС}} = 5.5 \text{ Гб}$;

$V_{\text{СПО}}$ – браузер Google Chrome, $V_{\text{СПО}} = 150 \text{ Мб}$);

Таким образом, суммарный объем внешней памяти составит:

$$V_{\text{ЖД}} = 5.5 \text{ Гб} + 150 \text{ Мб} \sim 5.6 \text{ Гб}.$$

Расчет объема ОЗУ

$$V_{\text{ОС}} = 1 \text{ Гб};$$

$V_{\text{БД}}$ – объем данных из базы, который может быть одновременно загружен в оперативную память (дадим ему оценку сверху в 10 Мб).

$V_{\text{браузера}}$ – ОЗУ, занимаемое браузером (оценим его сверху значением в 100 Мб).

Суммарные объемы ОЗУ составит:

$$V_{\text{ОЗУ}} = 1 \text{ Гб} + 10 \text{ Мб} + 100 \text{ Мб} \sim 1.1 \text{ Гб}.$$

3.3.2 Минимальные требования, предъявляемые к серверу

Для корректного функционирования системы необходимо:

- тип ЭВМ: x86-64 совместимый;
- объем ОЗУ – не менее 1.3 Гб;
- объем свободного дискового пространства – не менее 16.5 Гб;
- клавиатура или иное устройство ввода;
- мышь или иное манипулирующее устройство;
- дисплей с разрешением не менее 1024×768 пикселей;
- операционная система Windows 7 и выше;
- Python 3.

3.3.3 Минимальные требования, предъявляемые к клиенту

Для корректного функционирования системы необходимо:

- тип ЭВМ: x86-64 совместимый;
- объем ОЗУ – не менее 1.1 Гб;
- объем свободного дискового пространства – не менее 1.3 Гб;
- клавиатура или иное устройство ввода;
- мышь или иное манипулирующее устройство;
- дисплей с разрешением не менее 1024×768 пикселей;
- операционная система Windows 7 и выше;
- браузер Google Chrome.

ЗАКЛЮЧЕНИЕ

В процессе выполнения выпускной работы была разработана автоматизированная система сбора и анализа информации о новой коронавирусной инфекции, позволяющая получать, хранить и выдавать в различных форматах актуальные и исторические данные о распространении COVID-19 и принимаемых мерах противодействия.

В первом разделе были приведены основные понятия и определения предметной области, рассмотрено заболевание COVID-19, понятия иммунитета и коллективного иммунитета, вакцинации и мер по противодействию коронавирусу, приведены сравнительные характеристики систем-аналогов, на основании этого была сформулирована постановка задачи и основные требования к системе.

Во втором разделе была выбрана архитектура системы, разработана структура системы, спроектированы прототипы интерфейса пользователя, разработан информационно-логический проект системы по методологии UML, в состав которого вошли все канонические диаграммы, логическая модель данных, а также был выбран комплекс программных средств.

В третьем разделе был разработан и описан конечный интерфейс пользователя, выбран и обоснован комплекс технических средств, рассчитан объем занимаемой памяти, необходимый для работы системы, а также были установлены минимальные требования, предъявляемые к системе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Novel Coronavirus(2019-nCov) Situation report - 11 [Электронный ресурс] // Всемирная Организация Здравоохранения: [сайт]. URL: https://www.who.int/docs/default-source/coronaviruse/situation-reports/20200131-sitrep-11-ncov.pdf?sfvrsn=de7c0f7_2 (дата обращения: 06.12.2021).
- 2 WHO Director-General's opening remarks at the media briefing on COVID-19 - 11 March 2020 [Электронный ресурс] // Всемирная Организация Здравоохранения: [сайт]. URL: <https://www.who.int/director-general/speeches/detail/who-director-general-s-opening-remarks-at-the-media-briefing-on-covid-19---11-march-2020> (дата обращения: 06.12.2021).
- 3 Карпова, И.П. Основы баз данных [Текст] / И.П. Карпова. – Москва: Московский государственный институт электроники и математики (Технический университет), 2009. – 6 с.
- 4 Coronavirus disease 2019 (COVID-19) [Электронный ресурс] // BMJ: [сайт]. URL: <https://bestpractice.bmj.com/topics/en-gb/3000201> (дата обращения: 06.12.2021).
- 5 Fehr, A. Coronaviruses: An Overview of Their Replication and Pathogenesis [Текст] / A. R. Fehr, S. Perlman // Coronaviruses. Methods and protocols / ред. H.J. Maier, E. Bickerton, P. Britton. – Хатфилд, 2015. – с. 1.
- 6 Словарь медицинских терминов [Электронный ресурс] // Национальный медико-хирургический центр имени Н. И. Пирогова: [сайт]. URL: <https://www.pirogov-center.ru/patient/hospital/department/ophthalmology/terms.php> (дата обращения: 06.12.2021).
- 7 SARS-CoV-2 infection induces long-lived bone marrow plasma cells in humans / J.S. Turner, W. Kim, E. Kalaidina [и др.] // Nature. – 2021. – Vol. 595. URL: <https://www.nature.com/articles/s41586-021-03647-4> (дата обращения: 06.12.2021).
- 8 Вопросы и ответы: коллективный иммунитет, меры самоизоляции и COVID-19 [Электронный ресурс] // Всемирная Организация

Здравоохранения: [сайт]. URL: <https://www.who.int/ru/news-room/questions-and-answers/item/herd-immunity-lockdowns-and-covid-19> (дата обращения: 06.12.2021).

9 Иммунолог объяснил, чем опасно снижение коллективного иммунитета к COVID-19 [Электронный ресурс] // Москва24 [сайт]. URL: <https://www.m24.ru/news/obshchestvo/17032022/441347> (дата обращения: 06.12.2021).

10 Вакцина [Электронный ресурс] // Академик: [сайт]. URL: https://dic.academic.ru/dic.nsf/dic_fwords/37379/БАКЦИНА (дата обращения: 06.12.2021).

11 Olliaro, P. COVID-19 vaccine efficacy and effectiveness – the elephant (not) in the room / P. Olliaro, E. Torreele, M. Vaillant / DOI: 10.1016/S2666-5247(21)00069-0 // The Lancet. – 2021. – Vol. 2. URL: [https://www.thelancet.com/journals/lanmic/article/PIIS2666-5247\(21\)00069-0/fulltext](https://www.thelancet.com/journals/lanmic/article/PIIS2666-5247(21)00069-0/fulltext) (дата обращения: 06.12.2021).

12 Об определении порядка продления действия мер по обеспечению санитарно-эпидемиологического благополучия населения в субъектах Российской Федерации в связи с распространением новой коронавирусной инфекции (COVID-19) : указ Президента Российской Федерации [издан Президентом 11 мая 2020 г. № 316]. – Текст : непосредственный // Собрание законодательства Российской Федерации. – 2020. – № 20. – Ст. 3157

13 Запреты и ограничения на массовые мероприятия, услуги населению, туристические и другие услуги, а также их снятие [Электронный ресурс] // КонсультантПлюс: [сайт]. URL: http://www.consultant.ru/document/cons_doc_LAW_348585/c24fcecea7bd55b03d6189ddba7bdb7b46d1df6b/ (дата обращения: 06.12.2021).

14 Статистика [Электронный ресурс] // hmong.press: [сайт]. URL: <https://www.hmong.press/wiki/Statistica> (дата обращения: 06.12.2021).

15 Medcalc [Электронный ресурс]. URL: <https://www.medcalc.org/> (дата обращения: 06.12.2021).

16 What is your definition of Software Architecture? [Электронный ресурс] // Carnegie Mellon University: [сайт]. URL: https://resources.sei.cmu.edu/asset_files/FactSheet/2010_010_001_513810.pdf (дата обращения: 28.03.2022).

17 Зеленко Л.С. Методические указания к лабораторному практикуму по дисциплине «Программная инженерия». Самара: СГАУ, 2012.

18 Systems and software engineering - Vocabulary [Электронный ресурс] // the International Organisation for standartisation: [сайт]. URL: <https://www.iso.org/obp/ui/#iso:std:iso-iec-ieee:24765:ed-2:v1:en> (дата обращения: 28.03.2022).

19 Унифицированный язык моделирования UML. Базовые понятия диаграмм классов [Электронный ресурс] // Кафедра программного обеспечения автоматизированных систем Курганского Гос. Университета: [Сайт]. URL: http://it.kgsu.ru/UML/uml_0076.html (дата обращения: 28.03.2022).

20 Основные понятия баз данных [Электронный ресурс] // Южно-Уральский Государственный Университет: [сайт]. URL: http://inf.susu.ac.ru/Klinachev/lc_sga_26.htm (дата обращения: 28.03.2022).

21 Python programming language [Электронный ресурс] // irjet.net: [сайт]. URL: <https://www.irjet.net/archives/V6/i2/IRJET-V6I2367.pdf> (дата обращения: 04.01.2022).

22 Среда разработки PyCharm [Электронный ресурс] // jetbrains.com: [сайт]. URL: <https://www.jetbrains.com/ru-ru/pycharm/> (дата обращения: 28.03.2022).

23 PostgreSQL: About [Электронный ресурс] // postgresql.org: [сайт]. URL: <https://www.postgresql.org/about/> (дата обращения: 28.03.2022).

24 Диаграммы реализации [Электронный ресурс] // studopedia.ru: [сайт]. URL: https://studopedia.ru/7_19215_diagrammi-realizatsii.html (дата обращения: 05.01.2022).

25 Диаграммы развертывания [Электронный ресурс] // intellect.icu:

[сайт]. URL: <https://intellect.icu/diagramma-razvertyvaniya-deployment-diagram-uml-4831> (дата обращения: 05.01.2022).

26 Диаграммы компонентов [Электронный ресурс] // creately.com: [сайт]. URL: <https://creately.com/blog/ru/uncategorized-ru/ucebposobie/> (дата обращения: 05.01.2022).

27 SQL проектирование баз данных [Электронный ресурс] // ebook.su: [сайт]. URL: <https://ebook.su/db/sql-proektirovanie-baz-dannyh/> (дата обращения: 05.01.2022).

ПРИЛОЖЕНИЕ А

Руководство пользователя

А.1 Назначение системы

Данная система предназначена для сбора, анализа и получения пользователями информации о новой коронавирусной инфекции.

А.2 Условия работы системы

Для корректной работы системы необходимо наличие соответствующих программных и аппаратных средств.

1) Требования к техническому обеспечению сервера:

- тип ЭВМ: x86-64 совместимый;
- объем ОЗУ – не менее 1.3 Гб;
- объем свободного дискового пространства – не менее 16.5 Гб;
- клавиатура или иное устройство ввода;
- мышь или иное манипулирующее устройство;
- дисплей с разрешением не менее 1024×768 пикселей.

2) Требования к техническому обеспечению клиента:

- тип ЭВМ: x86-64 совместимый;
- объем ОЗУ – не менее 1.1 Гб;
- объем свободного дискового пространства – не менее 1.3 Гб;
- клавиатура или иное устройство ввода;
- мышь или иное манипулирующее устройство;
- дисплей с разрешением не менее 1024×768 пикселей;

3) Требования к программному обеспечению сервера:

- операционная система Windows 7 и выше;
- Python 3.

4) Требования к программному обеспечению клиента:

- операционная система Windows 7 и выше;
- браузер Google Chrome.

А.3 Работа с системой

При входе на сайт открывается главная страница (см. рисунок А.1). С помощью кнопок «Сформировать отчет», «Региональная статистика» и «Информация о сайте» пользователь может переходить к разделам сайта. Также на странице представлена актуальная информация о распространении COVID-19 в виде таблицы. Нажатие кнопки «Скачать данные» скачивает на компьютер пользователя JSON-файл, содержащий данные этой таблицы. Кнопки в футере страницы переводят к страницам разработчика в ВК и GitHub.

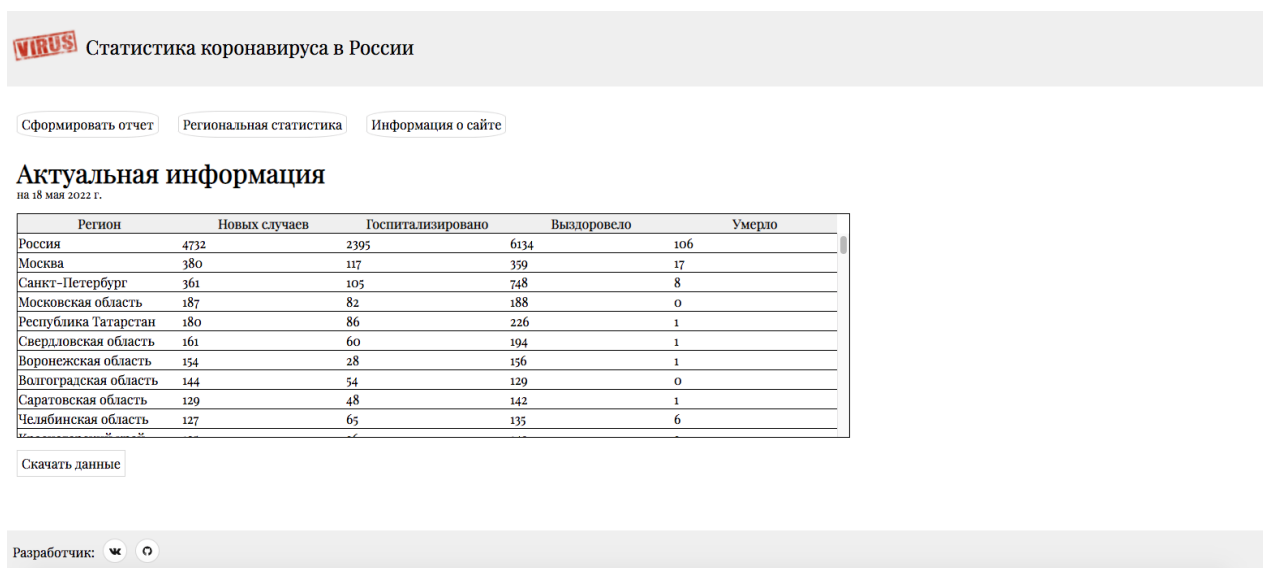


Рисунок А.1 – Главная страница сайта

А.3.1 Аналитические отчеты

Нажатие кнопки «Сформировать отчет» на главной странице переводит пользователя на страницу формирования отчета (см. рисунок А.2). Здесь пользователь может задать параметры отчета – выбрать регион, даты начала и конца отчетного периода, включить в отчет анализ мер, а также указать почту для отправки отчета. Когда все данные корректно введены – кнопка «Создать отчет» становится доступной к нажатию. Нажатие кнопки

скачивает сформированный по указанным параметрам отчет в формате документа Word на компьютер пользователя, а также, если был указан адрес, отправляет отчет на электронную почту. Для возвращения на главную страницу необходимо нажать кнопку «Назад» в шапке страницы.

Статистика коронавируса в России

Назад

Сформировать отчет:

Регион

Дата начала

Дата конца

☒ Включить в отчет анализ мер

☒ Отправить отчет на электронную почту

Адрес

Разработчик:

Рисунок А.2 – Страница формирования отчета

А.3.2 Региональная статистика

Нажатие кнопки «Региональная статистика» на главной странице переводит пользователя на страницу исторической региональной статистики (см. рисунок А.3).

Статистика коронавируса в России

Назад

Историческая статистика по регионам

Регион

Дата начала

Регион: Москва

Дата: 10 февраля 2022 г.

Новых случаев: 22747

Госпитализаций: 1374

Смертей: 84

Вакцинировано 1 компонентом: 7149637

Вакцинировано полностью: 6736593

Коллективный иммунитет: 63,2

Действующие ограничения:

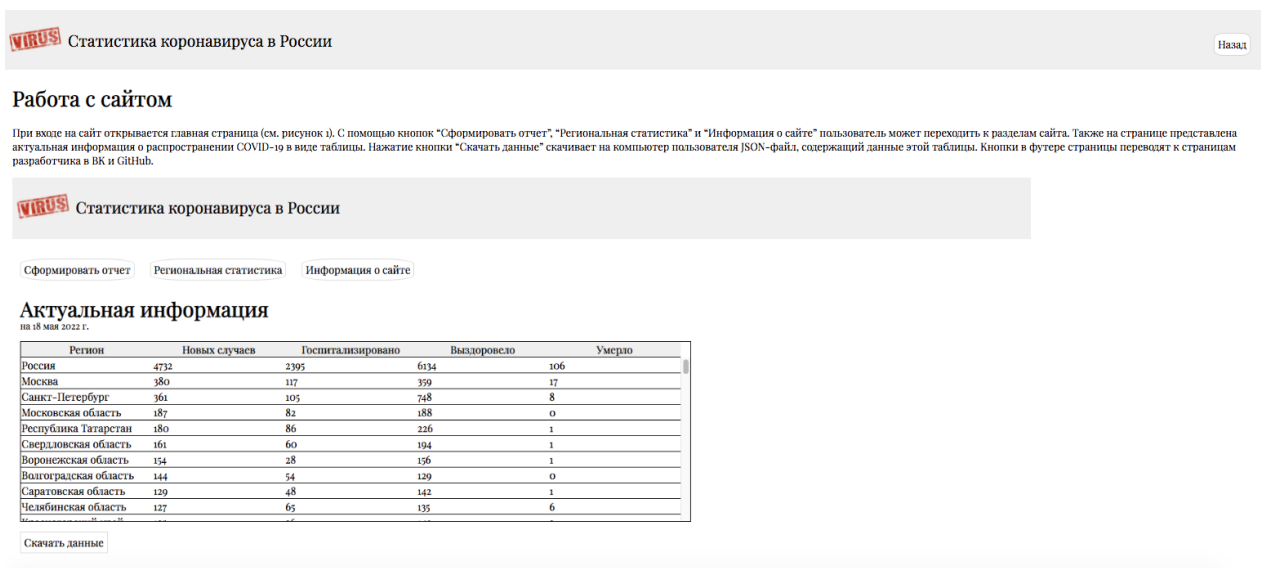
- Оказание услуг в МФЦ по предварительной записи или при наличии QR-кода, сертификата, ПИР-теста
- Введен масочный режим и социальное дистанцирование в общественном транспорте
- До 1 апреля 2022 года соблюдение режима самоизоляции для лиц старше 60 лет, а также лиц, имеющих хронические заболевания
- Введен удаленный режим работы для не менее 30% сотрудников организаций, лиц старше 60 лет, а также лиц, имеющих хронические заболевания
- Вакцинация 80% работников сферы услуг, городских госслужащих
- Введен масочный режим
- С 8 ноября 2021 года для лиц старше 60 лет и хронически больных граждан приостановлен бесплатный проезд в общественном транспорте (исключение для вакцинированных или переболевших в

Рисунок А.3 – Страница региональной статистики

Здесь пользователь может задать регион и дату. Нажатие кнопки «Получить данные» выдает необходимую информацию в блоке ниже. Для возвращения на главную страницу необходимо нажать кнопку «Назад» в шапке страницы.

А.3.3 Информация о сайте

Нажатие кнопки «Региональная статистика» на главной странице переводит пользователя на страницу исторической региональной статистики (см. рисунок А.4). Здесь пользователь может ознакомиться со справочной информацией и руководством пользователя.



Статистика коронавируса в России Назад

Работа с сайтом

При входе на сайт открывается главная страница (см. рисунок 1). С помощью кнопок "Сформировать отчет", "Региональная статистика" и "Информация о сайте" пользователь может переходить к разделам сайта. Также на странице представлена актуальная информация о распространении COVID-19 в виде таблицы. Нажатие кнопки "Скачать данные" скачивает на компьютер пользователя JSON-файл, содержащий данные этой таблицы. Кнопки в футере страницы переводят к страницам разработчика в VK и GitHub.

Статистика коронавируса в России

[Сформировать отчет](#) [Региональная статистика](#) [Информация о сайте](#)

Актуальная информация
на 18 мая 2022 г.

Регион	Новых случаев	Госпитализировано	Выздоровело	Умерло
Россия	4732	2395	6134	106
Москва	380	117	359	17
Санкт-Петербург	361	105	748	8
Московская область	187	82	188	0
Республика Татарстан	180	86	226	1
Свердловская область	161	60	194	1
Воронежская область	154	28	156	1
Волгоградская область	144	54	129	0
Саратовская область	129	48	142	1
Челябинская область	127	65	135	6

[Скачать данные](#)

Рисунок А.4 – Страница информации о сайте

ПРИЛОЖЕНИЕ Б

Код программы

```
from django.db import models

# Create your models here.
class Regions(models.Model):
    region = models.CharField(max_length=100, blank=False, verbose_name='Регион')

    def __str__(self):
        return self.region

    class Meta:
        verbose_name = 'Регион'
        verbose_name_plural = 'Регионы'
        db_table = 'regions'

class RegionalStats(models.Model):
    date = models.DateField(verbose_name='Дата') # todo заменить auto_now_add на дату из папса
    region = models.ForeignKey('Regions', on_delete=models.CASCADE, default=-1, verbose_name='id
региона')
    new_cases = models.IntegerField(verbose_name='Новые случаи')
    hospitalised = models.IntegerField(verbose_name='Госпитализировано')
    recovered = models.IntegerField(verbose_name='Вылечились')
    died = models.IntegerField(verbose_name='Умерло')
    vaccinated_first_component_cumulative = models.IntegerField(verbose_name='Вакцинировано 1
комп.')
    vaccinated_fully_cumulative = models.IntegerField(verbose_name='Вакцинировано 2 комп.')
    collective_immunity = models.DecimalField(max_digits=3, decimal_places=1,
verbose_name='Коллективный иммунитет')

    def __str__(self):
        return f'Дата: {self.date} '\
            f'Регион: {self.region.region} '\
            f'Новых случаев: {self.new_cases} '\
            f'Госпитализировано: {self.hospitalised} '\
            f'Вылечились: {self.recovered} '\
            f'Умерло: {self.died} '\
            f'Вакцинировано первым компонентом: {self.vaccinated_first_component_cumulative} '\
            f'Вакцинировано полностью: {self.vaccinated_fully_cumulative} '\
            f'Коллективный иммунитет: {self.collective_immunity}'

    class Meta:
        verbose_name = 'Региональная статистика'
        verbose_name_plural = 'Региональная статистика'
        db_table = 'regional_stats'
        ordering = ['-date', 'region']
        unique_together = ['date', 'region']

class Restrictions(models.Model):
    description = models.TextField(unique=True)

    def __str__(self):
        return self.description

    class Meta:
        verbose_name = 'Ограничение'
        verbose_name_plural = 'Ограничения'
```

```

class RegionalRestrictions(models.Model):
    region = models.ForeignKey('Regions', on_delete=models.CASCADE, default=-1, verbose_name='id
региона')
    date = models.DateField(auto_now_add=True, verbose_name='Дата')
    restriction = models.ForeignKey('Restrictions', on_delete=models.CASCADE, default=-1,
verbose_name='Ограничение')

    def __str__(self):
        return f'Регион: {self.region}' \
            f'Дата: {self.date}' \
            f'Ограничение: {self.restriction}'

class Meta:
    verbose_name = 'Региональное ограничение'
    verbose_name_plural = 'Региональные ограничения'
    db_table = 'regional_restrictions'
    ordering = ['-date', 'region']
    unique_together = ['date', 'region', 'restriction']

# todo Возможно не нужна из-за unique together?
class LastParsed(models.Model):
    parse_datetime = models.DateTimeField(auto_now=True, verbose_name='Время последней проверки')
    parsed_info_datetime = models.DateField(verbose_name='Время последних взятых данных')
    parse_success = models.BooleanField(verbose_name='Парсинг прошел успешно', default=False)

    def __str__(self):
        if self.parse_success:
            return f'Последний парсинг (успешный) - {self.parse_datetime}, последние данные -
{self.parsed_info_datetime}'
        else:
            return f'Последний парсинг не прошел в {self.parse_datetime}, последние данные -
{self.parsed_info_datetime}'

class Meta:
    verbose_name = 'Последний парсинг'
    verbose_name_plural = 'Последний парсинг'
    db_table = 'last_parsed'

from django.shortcuts import render, redirect
import json
from django.http import HttpResponse
import os
from datetime import datetime

from .models import RegionalStats, LastParsed

# Create your views here.
def index(request):
    context = {}
    latest_date = LastParsed.objects.get().parsed_info_datetime
    regional_stats_list = RegionalStats.objects.filter(date=latest_date).order_by('-new_cases')
    # print(regions)
    # print(regional_stats_list)
    # regional_stats_list = list(range(80))
    context['regional_stats_list'] = regional_stats_list
    regional_stats_columns = ['region', 'new_cases', 'hospitalised', 'recovered', 'died']
    context['regional_stats_columns'] = regional_stats_columns
    context['actual_date'] = latest_date

```

```

return render(request, 'stats/index.html', context)

def download_current(request):
    print('Скачивание')
    # print(os.getcwd())
    latest_date = LastParsed.objects.get().parsed_info_datetime
    # print(latest_date)
    # print(type(latest_date))
    regional_stats_list = RegionalStats.objects.filter(date=latest_date)
    result = {'stats': []}
    filepath = f'static/files/jsons/current_{latest_date}.json'
    for regional_stats in regional_stats_list:
        result['stats'].append({
            'date': str(regional_stats.date),
            'region': regional_stats.region.region,
            'new_cases': regional_stats.new_cases,
            'hospitalised': regional_stats.hospitalised,
            'recovered': regional_stats.recovered,
            'died': regional_stats.died,
            'vaccinated_first_component_cumulative': regional_stats.vaccinated_first_component_cumulative,
            'vaccinated_fully_cumulative': regional_stats.vaccinated_fully_cumulative,
            'collective_immunity': float(regional_stats.collective_immunity),
        })
    # print(result)
    if not os.path.isfile(filepath):
        with open(filepath, 'w') as file:
            json.dump(result, file, indent=2, ensure_ascii=False)
    with open(filepath, 'rb') as file:
        response = HttpResponse(file.read())
        response['Content-Disposition'] = "attachment; filename=%s" % filepath
    return response

from django.shortcuts import render

from stats.models import Regions, RegionalStats, RegionalRestrictions
from datetime import datetime

# Create your views here.

def regions_page(request):
    context = {'region': "",
               'date': "",
               'new_cases': "",
               'hospitalised': "",
               'died': "",
               'vac1': "",
               'vac_full': "",
               'immunity': "",
               'restrictions': ""}
    inregion = request.GET.get('region', "")
    print(inregion)
    indate = request.GET.get('date', "")
    context['inregion'] = inregion
    context['indate'] = indate
    # print(f"region -'{region}', date = '{date}'")
    if inregion != "" and indate != "":
        date = datetime.strptime(indate, '%Y-%m-%d').date()
        region = Regions.objects.filter(region=inregion)
        if region.count() == 0:

```

```

        context = {'region': inregion,
                    'date': date,
                    'new_cases': 'Нет данных',
                    'hospitalised': 'Нет данных',
                    'died': 'Нет данных',
                    'vac1': 'Нет данных',
                    'vac_full': 'Нет данных',
                    'immunity': 'Нет данных',
                    'restrictions': "",
                    'inregion': region,
                    'indate': indate,
                    }
    else:
        region = region[0]
        stats = RegionalStats.objects.filter(date=date, region=region)
        if stats.count() == 0:
            context = {'region': inregion,
                        'date': date,
                        'new_cases': 'Нет данных',
                        'hospitalised': 'Нет данных',
                        'died': 'Нет данных',
                        'vac1': 'Нет данных',
                        'vac_full': 'Нет данных',
                        'immunity': 'Нет данных',
                        'restrictions': "",
                        'inregion': region,
                        'indate': indate,
                        }
        else:
            stats = stats[0]
            context['region'] = region
            context['date'] = date
            context['new_cases'] = stats.new_cases
            context['hospitalised'] = stats.hospitalised
            context['died'] = stats.died
            context['vac1'] = stats.vaccinated_first_component_cumulative
            context['vac_full'] = stats.vaccinated_fully_cumulative
            context['immunity'] = stats.collective_immunity
            restrictions = RegionalRestrictions.objects.filter(date=date, region=region)
            restrictions = list(set([restr.restriction.description for restr in restrictions]))
            context['restrictions'] = restrictions
            print(restrictions)
    return render(request, 'regions/regions_page.html', context)

```

```

import requests
import logging
from re import sub
from django.db import IntegrityError
from bs4 import BeautifulSoup
from bs4.element import Tag
from json import loads
from stats.models import RegionalStats, Regions, LastParsed, \
    Restrictions, RegionalRestrictions
from datetime import datetime

```

```

stop_coronavirus_url = 'https://xn--80aesfpebagmfb1c0a.xn--p1ai/information/'
unicode_symbols = {
    "\u0410": 'A',
    "\u0411": 'B',
    "\u0412": 'B',
    "\u0413": 'T',

```



```

'\u0414': 'Д',
'\u0415': 'Е',
'\u0416': 'Ж',
'\u0417': 'З',
'\u0418': 'И',
'\u0419': 'Й',
'\u041a': 'К',
'\u041b': 'Л',
'\u041c': 'М',
'\u041d': 'Н',
'\u041e': 'О',
'\u041f': 'П',
'\u0420': 'Р',
'\u0421': 'С',
'\u0422': 'Т',
'\u0423': 'У',
'\u0424': 'Ф',
'\u0425': 'Х',
'\u0426': 'Ц',
'\u0427': 'Ч',
'\u0428': 'Ш',
'\u0429': 'Щ',
'\u042a': 'Ъ',
'\u042b': 'Ы',
'\u042c': 'Ь',
'\u042d': 'Э',
'\u042e': 'Ю',
'\u042f': 'Я',
'\u0430': 'а',
'\u0431': 'б',
'\u0432': 'в',
'\u0433': 'г',
'\u0434': 'д',
'\u0435': 'е',
'\u0436': 'ж',
'\u0437': 'з',
'\u0438': 'и',
'\u0439': 'й',
'\u043a': 'к',
'\u043b': 'л',
'\u043c': 'м',
'\u043d': 'н',
'\u043e': 'о',
'\u043f': 'п',
'\u0440': 'р',
'\u0441': 'с',
'\u0442': 'т',
'\u0443': 'у',
'\u0444': 'ф',
'\u0445': 'х',
'\u0446': 'ц',
'\u0447': 'ч',
'\u0448': 'ш',
'\u0449': 'щ',
'\u044a': 'ъ',
'\u044b': 'ы',
'\u044c': 'ь',
'\u044d': 'э',
'\u044e': 'ю',
'\u044f': 'я'
}
symbols_to_replace = {
    '\u0414': 'Д',

```

```

        '\r': ' ',
        '\n': '\n',
        '<li>': ' ',
        '</li>': ' ',
        '<ul>': ' ',
        '</ul>': ' '
    }
    months = {'января': '01',
              'февраля': '02',
              'марта': '03',
              'апреля': '04',
              'мая': '05',
              'июня': '06',
              'июля': '07',
              'августа': '08',
              'сентября': '09',
              'октября': '10',
              'ноября': '11',
              'декабря': '12'}
    country_keys_change = {'sickChange': 'sick_incr',
                           'hospitalized': 'hospitalized',
                           'healedChange': 'healed_incr',
                           'diedChange': 'died',
                           'vaccineFirst': 'first',
                           'vaccineSecond': 'second'}
    restriction_changes = {
        'и загрузить результат на портал госуслуг. До получения результатов теста необходимо соблюдать режим самоизоляции.': 'и загрузить результат на портал госуслуг; До получения результатов теста необходимо соблюдать режим самоизоляции.',
        'т.ч.': 'т.ч.',
        'им. ': 'им ',
        '.00': ':00',
        'кв.м.': 'кв м',
        'др.': 'др',
        'г.': 'г',
        'соц.': 'соц'
    }
}

```

```

def __get_stopcoronavirus_rf_page__():
    """
    Отправка реквеста на стопкоронавирус.рф/information/
    :return: Текст страницы, либо None при возникновении ошибки
    :raises Exception: e - ошибка для дальнейшей обработки
    """

```

```

    try:
        return requests.get(stop_coronavirus_url).text
    except requests.ConnectionError as e:
        print('При отправке запроса произошла ошибка соединения')
        raise e
    except requests.RequestException as e:
        print('При отправке запроса произошло исключение')
        raise e
    except Exception as e:
        print('При отправке запроса произошла неизвестная ошибка')
        raise e

```

```

def parseregions(page_text):
    return __parse_regions_stopcoronavirus_rf_page__(page_text)

```

```

def __parse_regions_stopcoronavirus_rf_page__(page_text: str) -> Tag:

```

```

"""Выделяет из текста страницы тег регионов

:param page_text: Текст страницы стопкоронавирус.рф/information/
:param type: regions для получения данных по регионам, country - для России
:return: Тэг cv-spread-overview, содержащий актуальную информацию либо None в случае ошибки
:raises Exception: e - ошибка для дальнейшей обработки
"""

if page_text is None:
    raise ValueError('Текст страницы is none')
try:
    soup = BeautifulSoup(page_text, 'html.parser')
    tag = soup.find_all('cv-spread-overview')[0]
    tag.findChild(recursive=False).decompose()
    return tag
except Exception as e:
    print('При парсинге регионов из текста страницы произошла ошибка:' + str(e))
    raise e

def __get_strings_from_tag__(tag: Tag):
    """
    Получение необработанных списков данных из тэга
    :param tag: Тэг cv-spread-overview, содержащий актуальную информацию
    :return: tuple списков с информацией (в строковом формате без обработки) либо None в случае
    ошибки
    :raises Exception: e - ошибка для дальнейшей обработки
    """
    if tag is None:
        raise ValueError('Тег данных is none')
    try:
        spread_data = tag.get(':spread-data')

        # Данные о степенях изоляции, возможно понадобятся
        # todo Добавить либо удалить
        # isolation_data = div.get(':isolation-data')
        # covid_free_states = div.get(':covid-free-states')

        global_stat_date = tag.get('global-stat-date')
        return spread_data, global_stat_date
    except Exception as e:
        print("При формировании листов произошла ошибка")
        raise e

def __clean_string__(string: str) -> str:
    """
    Очистка строки от символов HTML, лишних пробелов, а также замена символов в кодировке
    Юникод
    :param string: строка для замены
    :return: res_string: очищенная строка либо None в случае ошибки
    :raises Exception: e - ошибка для дальнейшей обработки
    """
    try:
        res_string = string
        for unicode_char, char in unicode_symbols.items():
            res_string = res_string.replace(unicode_char, char)
        for to_replace, replace_by in symbols_to_replace.items():
            res_string = res_string.replace(to_replace, replace_by)
        res_string = " ".join(res_string.split())
        return res_string
    except Exception as e:
        print(f'Произошла ошибка при парсинге строки: {string}')
        raise e

```

```
def __eval_string__(string: str):
    """
    Перевод данных из строкового формата в массив
    :param string: Строка с данными
    :return: Результат перевода либо None в случае ошибки
    :raises Exception: e - ошибка для дальнейшей обработки
    """
    try:
        result = loads(string)
        return result
    except Exception as e:
        print('При переводе информации из строкового типа произошла ошибка')
        raise e
```

```
def __parse_date__(date_str: str) -> str:
    """Переводит дату в необходимый формат

    Из '11 марта 14:20' в '11.03.*актуальный год*'

    :param date_str: дата в формате парсинга
    :return: дата в новом формате
    """
    try:
        split = date_str.split(' ')
        return f'{datetime.today().year}-{months[split[1]]}-{split[0]}'
    except Exception as e:
        print('При переводе даты в другой формат произошла ошибка')
        raise e
```

```
def __parse_country_stopcoronavirus_rf_page__(page_text: str) -> dict:
    """Выделяет из текста страницы информацию по стране
```

Проводит весь цикл работы от текста до словаря, так как данные по стране парсятся иначе чем региональные

```
:param page_text: Текст страницы стопкоронавирус.рф/information/
:return: Словарь с данными о России
"""
if page_text is None:
    raise ValueError('Текст страницы is none')
try:
    country_stats = {'title': 'Россия'}
    soup = BeautifulSoup(page_text, 'html.parser')
    # print(soup)
    spread_tag = soup.find_all('cv-stats-virus')[0]
    stats_data = spread_tag.get(':stats-data')
    stats_data = __eval_string__(stats_data)
    for k, v in stats_data.items():
        stats_data[k] = int(v.replace(',', ''))
    for k, v in country_keys_change.items():
        country_stats[v] = stats_data[k]
    collective_immunity = soup.find_all('h3')[2]
    collective_immunity = collective_immunity.text
    collective_immunity = float(collective_immunity.rstrip('%').replace(',', ''))
    country_stats['immune_percent'] = collective_immunity
    country_stats['died_incr'] = country_stats['died']
    print(country_stats)
    return country_stats
except Exception as e:
```

```

print('При парсинге страны из текста страницы произошла ошибка.')
raise e

```

```

def __create_regional_stats_list__(dicts: list, date: str) -> list:
    """Формирует список статистики типа данных модели

    :param dicts: массив словарей со статистикой (формат парсинга)
    :param date: дата данных
    :return: список статистики типа данных RegionalStats
    """
    try:
        regional_stats = []
        # print('Создание списка')
        for region_dict in dicts:
            # print(region_dict)
            # print(region_dict['isolation'])
            region_title = region_dict['title'].strip(' ')
            region = Regions.objects.get(region=region_title)
            new_cases = region_dict['sick_incr']
            hospitalised = region_dict['hospitalized']
            recovered = region_dict['healed_incr']
            died = region_dict['died_incr']
            vaccinated_first_component_cumulative = region_dict['first']
            vaccinated_fully_cumulative = region_dict['second']
            collective_immunity = region_dict['immune_percent']
            stats = RegionalStats(date=date, region=region, new_cases=new_cases, hospitalised=hospitalised,
                                   recovered=recovered, died=died,
                                   vaccinated_first_component_cumulative=vaccinated_first_component_cumulative,
                                   vaccinated_fully_cumulative=vaccinated_fully_cumulative,
                                   collective_immunity=collective_immunity)
            regional_stats.append(stats)
        if len(regional_stats) != 86:
            raise ValueError('Не все регионы прошли парсинг')
        return regional_stats
    except Exception as e:
        print('При создании списка статистики произошла ошибка')
        raise e

```

```

def __create_regional_restrictions_list__(dicts: list, date: str) -> list:
    try:
        regional_stats = []
        for region_dict in dicts:
            print(region_dict)
            region_title = region_dict['title'].strip(' ')
            region = Regions.objects.get(region=region_title)
            print(region_title)
            print(region)
            # print(region_dict)
            # print(region_dict['isolation'])
            # print()
            restrictions = region_dict['isolation']['descr']
            if restrictions is None:
                continue
            print(restrictions)
            print()
            restrictions = sub("<a.*>", "", restrictions)
            restrictions = sub("<.*>", "", restrictions)
            for k, v in restriction_changes.items():
                restrictions = restrictions.replace(k, v)
            for item in restrictions.split('.'):
                restriction = item.strip(' ')

```

```

        if restriction == "":
            continue
        try:
            Restrictions(description=restriction).save()
        except IntegrityError as e:
            pass
        finally:
            restriction = Restrictions.objects.get(description=restriction)
            regional_stats.append(RegionalRestrictions(region=region, date=date, restriction=restriction))
    # print(len(regional_stats))
    # print(regional_stats)
    return regional_stats
except Exception as e:
    print('При создании списка ограничений произошла ошибка')
    raise e

def __save_new_stats__(stats: list, restrictions: list, date: str):
    try:
        for region_restrictions in restrictions:
            try:
                region_restrictions.save()
            except IntegrityError as e:
                pass
        for region_stats in stats:
            try:
                region_stats.save()
            except IntegrityError as e:
                pass
        parse_info = LastParsed.objects.get()
        parse_info.parse_success = True
        parse_info.parsed_info_datetime = stats[0].date
        # print(parse_info)
        parse_info.save()
    except Exception as e:
        print('При сохранении ограничений произошла ошибка')
        raise e

def update() -> bool:
    """
    Проводит весь процесс взятия данных со стопкоронавирус.рф, их обработки и занесения в базу
    данных.
    :return bool: Успешность выполнения парсинга
    """
    print("Начат процесс сверки данных с стопкоронавирус.рф")
    try:
        page_text = __get_stopcoronavirus_rf_page__()
        regions_tag = __parse_regions_stopcoronavirus_rf_page__(page_text)
        spread_data, global_stat_date = __get_strings_from_tag__(regions_tag)
        global_stat_date = __parse_date__(global_stat_date)
        spread_data = __clean_string__(spread_data)
        spread_data = __eval_string__(spread_data)
        restrictions_list = __create_regional_restrictions_list__(spread_data, global_stat_date)
        spread_data.append(__parse_country_stopcoronavirus_rf_page__(page_text).copy())
        stats_list = __create_regional_stats_list__(spread_data, global_stat_date)
        __save_new_stats__(stats_list, restrictions_list, global_stat_date)
        print("Сверка данных успешно завершена")
        return True
    except Exception as e:
        print('Ошибка в функции db_updater.updater.update(): ' + str(e))
        parse_info = LastParsed.objects.get()
        parse_info.parse_datetime = datetime.now()

```

```

        parse_info.parse_success = False
        parse_info.save()
        return False

if __name__ == '__main__':
    update()

from django.shortcuts import redirect
from django.http import HttpResponse, HttpRequest
from db_updater.updater import update

# Create your views here.
def parse(request: HttpRequest):
    print(request.method)
    print('Начат процесс парсинга')
    print(request.path)
    print(request.path_info)
    update()
    return redirect('/admin/stats/lastparsed/')

"""
Django settings for coronavirus_web project.

Generated by 'django-admin startproject' using Django 4.0.2.

For more information on this file, see
https://docs.djangoproject.com/en/4.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/4.0/ref/settings/
"""

from pathlib import Path
import os

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/4.0/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-&)7woo@s2#uc=jokp^@3_$%*txy7z+3&h969h9sxeb0-zi-m#b'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',

```

```

'django.contrib.staticfiles',
'stats.apps.StatsConfig',
'index_redirect.apps.IndexRedirectConfig',
'reports.apps.ReportsConfig',
'site_info.apps.SiteInfoConfig',
'db_updater.apps.DbUpdaterConfig',
'regions.apps.RegionsConfig'
]

MIDDLEWARE = [
'django.middleware.security.SecurityMiddleware',
'django.contrib.sessions.middleware.SessionMiddleware',
'django.middleware.common.CommonMiddleware',
'django.middleware.csrf.CsrfViewMiddleware',
'django.contrib.auth.middleware.AuthenticationMiddleware',
'django.contrib.messages.middleware.MessageMiddleware',
'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'coronavirus_web.urls'

TEMPLATES = [
{
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    'DIRS': [],
    'APP_DIRS': True,
    'OPTIONS': {
        'context_processors': [
            'django.template.context_processors.debug',
            'django.template.context_processors.request',
            'django.contrib.auth.context_processors.auth',
            'django.contrib.messages.context_processors.messages',
        ],
    },
},
]

WSGI_APPLICATION = 'coronavirus_web.wsgi.application'

# Database
# https://docs.djangoproject.com/en/4.0/ref/settings/#databases

# Тестовая SQLite база
"""DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db backup.sqlite3',
    }
}"""

# Используемая PostgreSQL база
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'coviddb',
        'USER': 'coronaweb',
        'PASSWORD': 'coronawebpass',
        'HOST': 'localhost',
        'PORT': '5433'
    }
}

```



```

# Password validation
# https://docs.djangoproject.com/en/4.0/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/4.0/topics/i18n/

LANGUAGE_CODE = 'ru'

TIME_ZONE = 'Europe/Samara'

USE_I18N = True

USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/4.0/howto/static-files/

STATIC_URL = 'static/'

MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
MEDIA_URL = '/media/'
STATIC_URL = 'static/'

# Default primary key field type
# https://docs.djangoproject.com/en/4.0/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

STATICFILES_DIRS = [
    BASE_DIR / "static",
]

```